

Практическая работа №2.3.

Оптимизация доступа к данным при помощи индексации и секционирования

План выполнения работы

1. Изучить лекционный материал, текущее задание, и создать необходимые таблицы на основе данных, использованных в предыдущей практической работе.

Для выполнения практической работы использовались таблицы с данными, созданные в ходе выполнения работы №2:

1) Случайно распределенную (Таблица 1 - companies_randomly).

```
CREATE TABLE companies_randomly (  
    LIKE companies  
) DISTRIBUTED RANDOMLY;  
  
INSERT INTO companies_randomly SELECT * FROM companies;
```

2) Распределенную по полю id (Таблица 2 - companies_by_id).

```
CREATE TABLE companies_by_id (  
    LIKE companies  
) DISTRIBUTED BY (id);  
  
INSERT INTO companies_by_id SELECT * FROM companies;
```

3) Распределенную по полю region_id (строчный способ хранения) (Таблица 3 - companies_by_region_id_row).

```
CREATE TABLE companies_by_region_id_row (  
    LIKE companies  
)  
WITH (appendoptimized=true, orientation=row)  
DISTRIBUTED BY (region_id);  
  
INSERT INTO companies_by_region_id_row SELECT * FROM companies;
```

4) Распределенную по полю region_id (колоночный способ хранения) (Таблица 4 - companies_by_region_id_col).

```
CREATE TABLE companies_by_region_id_col (  
    LIKE companies  
)  
WITH (appendoptimized=true, orientation=column)
```

```
DISTRIBUTED BY (region_id);

INSERT INTO companies_by_region_id_col SELECT * FROM companies;
```

Соберем статистику:

```
ANALYZE companies_randomly;
ANALYZE companies_by_id;
ANALYZE companies_by_region_id_row;
ANALYZE companies_by_region_id_col;
```

2. Выполнить запросы из Практическая работа 3.sql. для Таблицы 1, используя оба оптимизатора. Сравните планы исполнения и метрические показатели. Результаты оформите в виде таблицы.

1 запрос:

```
-- 1 запрос
EXPLAIN analyze
select * from companies_randomly
where region_id = '77';

SET optimizer = off;
EXPLAIN analyze
select * from companies_randomly
where region_id = '77';

SET optimizer = ON;
```

Используя GPORCA:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..482.28 rows=25874 width=291) (actual time=0.223..19.514 rows=25876 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..448.47 rows=12937 width=291) (actual time=0.005..10.184 rows=12989 loops=1)
3	Filter: (region_id = 77)
4	Planning time: 2.083 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Pivotal Optimizer (GPORCA)
9	Execution time: 20.824 ms

Используя Postgres Query Optimizer:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=25874 width=291) (actual time=0.332..19.660 rows=25876 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..2262.00 rows=12937 width=291) (actual time=0.008..12.009 rows=12989 loops=1)
3	Filter: (region_id = 77)
4	Planning time: 0.134 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Postgres query optimizer
9	Execution time: 21.067 ms

2 запрос:

```
-- 2 запрос
EXPLAIN analyze
select * from companies_randomly
where region_id = '29';

SET optimizer = off;
EXPLAIN analyze
select * from companies_randomly
where region_id = '29';

SET optimizer = ON;
```

Используя GPORCA:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..442.06 rows=389 width=290) (actual time=0.999..9.774 rows=472 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..441.55 rows=195 width=290) (actual time=0.039..9.053 rows=242 loops=1)
3	Filter: (region_id = 29)
4	Planning time: 11.637 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Pivotal Optimizer (GPORCA)
9	Execution time: 10.172 ms

Используя Postgres Query Optimizer:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=389 width=290) (actual time=1.349..9.212 rows=472 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..2262.00 rows=195 width=290) (actual time=0.057..8.716 rows=242 loops=1)
3	Filter: (region_id = 29)
4	Planning time: 0.150 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Postgres query optimizer
9	Execution time: 9.559 ms

3 запрос:

```
-- 3 запрос
```

```
EXPLAIN analyze
select * from companies_randomly
where region_id = '61';

SET optimizer = off;
EXPLAIN analyze
select * from companies_randomly
where region_id = '61';

SET optimizer = ON;
```

Используя GPORCA:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..444.08 rows=1674 width=290) (actual time=0.696..12.279 rows=1735 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..441.90 rows=837 width=290) (actual time=0.014..11.113 rows=870 loops=1)
3	Filter: (region_id = 61)
4	Planning time: 2.913 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Pivotal Optimizer (GPORCA)
9	Execution time: 12.733 ms

Используя Postgres Query Optimizer:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=1674 width=290) (actual time=0.831..13.312 rows=1735 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..2262.00 rows=837 width=290) (actual time=0.020..9.470 rows=870 loops=1)
3	Filter: (region_id = 61)
4	Planning time: 0.259 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Postgres query optimizer
9	Execution time: 13.940 ms

Результат в виде таблицы:

Номер запроса	Оптимизатор	Время планирования	Время выполнения	Использованной памяти	Оценка строк/Реальная цифра
1	GPORCA	2.083 ms	20.824 ms	128000kB	25874/25876
1	Postgres Query Optimizer	0.134 ms	21.067 ms	128000kB	25874/25876
2	GPORCA	11.637 ms	10.172 ms	128000kB	389/472
2	Postgres	0.150 ms	9.559 ms	128000kB	389/472

	Query Optimizer				
3	GPORCA	2.913 ms	12.733 ms	128000kB	1674/1735
3	Postgres Query Optimizer	0.259 ms	13.940 ms	128000kB	1674/1735

Вывод:

При выполнении простых запросов с полным сканированием таблицы (Seq Scan) оба оптимизатора показывают почти одинаковое время выполнения (Execution time). Основное различие заключается в модели расчёта cost: GPORCA использует MPP-optimized cost model, что даёт значительно более низкие значения cost (в 4-5 раз), ведь она учитывает MPP модель, но эти значения не подлежат сравнению с cost от PostgreSQL оптимизатора. PostgreSQL оптимизатор значительно быстрее планирует запросы, что делает его предпочтительным для простых запросов. GPORCA лучше подходит для сложных аналитических запросов с JOIN, подзапросами и агрегациями.

3. Создайте btree-индекс для Таблицы 1 по полю region_id. Повторите шаги п.2, сравните получившиеся планы исполнения с планами из п.2. Результаты оформите в виде таблицы. Удалите созданный индекс.

Создадим индекс:

```
CREATE INDEX idx_btree_region_id ON companies_randomly USING btree
(region_id);
ANALYZE companies_randomly;
```

1 запрос:

```
-- 1 запрос
EXPLAIN analyze
select * from companies_randomly
where region_id = '77';

SET optimizer = off;
EXPLAIN analyze
select * from companies_randomly
where region_id = '77';
```

```
SET optimizer = ON;
```

Используя GPORCA:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..64.19 rows=25810 width=292) (actual time=0.939..14.164 rows=25876 loops=1)
2	-> Index Scan using idx_btree_region_id on companies_randomly (cost=0.00..23.34 rows=12905 width=292) (actual time=0.347..5.263 rows=12989 loops=1)
3	Index Cond: (region_id = 77)
4	Planning time: 11.685 ms
5	(slice0) Executor memory: 126K bytes.
6	(slice1) Executor memory: 114K bytes avg x 2 workers, 114K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Pivotal Optimizer (GPORCA)
9	Execution time: 15.367 ms

Используя Postgres Query Optimizer:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=25794 width=290) (actual time=0.564..27.404 rows=25876 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..2262.00 rows=12897 width=290) (actual time=0.015..15.835 rows=12989 loops=1)
3	Filter: (region_id = 77)
4	Planning time: 0.395 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Postgres query optimizer
9	Execution time: 29.280 ms

2 запрос:

```
-- 2 запрос
EXPLAIN analyze
select * from companies_randomly
where region_id = '29';

SET optimizer = off;
EXPLAIN analyze
select * from companies_randomly
where region_id = '29';
SET optimizer = ON;
```

Используя GPORCA:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..6.87 rows=390 width=290) (actual time=0.287..0.600 rows=472 loops=1)
2	-> Index Scan using idx_btree_region_id on companies_randomly (cost=0.00..6.26 rows=195 width=290) (actual time=0.039..0.191 rows=242 loops=1)
3	Index Cond: (region_id = 29)
4	Planning time: 17.108 ms
5	(slice0) Executor memory: 126K bytes.
6	(slice1) Executor memory: 114K bytes avg x 2 workers, 114K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Pivotal Optimizer (GPORCA)
9	Execution time: 0.944 ms

Используя Postgres Query Optimizer:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=390 width=290) (actual time=1.659..13.852 rows=472 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..2262.00 rows=195 width=290) (actual time=0.105..12.943 rows=242 loops=1)
3	Filter: (region_id = 29)
4	Planning time: 0.197 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Postgres query optimizer
9	Execution time: 14.336 ms

3 запрос:

```
-- 3 запрос
EXPLAIN analyze
select * from companies_randomly
where region_id = '61';

SET optimizer = off;
EXPLAIN analyze
select * from companies_randomly
where region_id = '61';

SET optimizer = ON;
```

Используя GPORCA:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..10.03 rows=1800 width=290) (actual time=0.612..3.577 rows=1735 loops=1)
2	-> Index Scan using idx_btree_region_id on companies_randomly (cost=0.00..7.20 rows=900 width=290) (actual time=0.110..1.324 rows=870 loops=1)
3	Index Cond: (region_id = 61)
4	Planning time: 4.116 ms
5	(slice0) Executor memory: 126K bytes.
6	(slice1) Executor memory: 114K bytes avg x 2 workers, 114K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Pivotal Optimizer (GPORCA)
9	Execution time: 4.153 ms

Используя Postgres Query Optimizer:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=1800 width=290) (actual time=0.700..10.520 rows=1735 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..2262.00 rows=900 width=290) (actual time=0.016..8.790 rows=870 loops=1)
3	Filter: (region_id = 61)
4	Planning time: 0.265 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Postgres query optimizer
9	Execution time: 11.002 ms

Результат в виде таблицы:

Номер запроса	Оптимизатор	Время планирования	Время выполнения	Использованной памяти	Оценка строк/Реальная цифра	Тип сканирования
1	GPORCA	11.685 ms	15.367 ms	128000 kB	25810/25876	Index
1	Postgres Query Optimizer	0.395 ms	29.280 ms	128000 kB	25794/25876	Seq
2	GPORCA	17.108 ms	0.944 ms	128000 kB	390/472	Index
2	Postgres Query Optimizer	0.197 ms	14.336 ms	128000 kB	390/472	Seq
3	GPORCA	4.116 ms	4.153 ms	128000 kB	1800/1735	Index
3	Postgres Query Optimizer	0.265 ms	11.002 ms	128000 kB	1800/1735	Seq

Вывод:

Создание B-tree индекса по полю region_id позволило GPORCA использовать Index Scan для всех трёх запросов, что привело к значительному ускорению выполнения. PostgreSQL Optimizer несмотря на наличие индекса продолжил использовать Seq Scan, демонстрируя худшую производительность (в 2-15 раз медленнее). Это подтверждает преимущество GPORCA в MPP-среде Greenplum при работе с индексами.

Удалим индекс:

```
DROP INDEX idx_btree_region_id;
x_btree_region_id
```

4. Создайте bitmap-индекс для Таблицы 1 по полю region_id. Повторите шаги п.3, сравните получившиеся планы исполнения с планами из п.3. Результаты оформите в виде таблицы. Удалите созданный индекс.

Создадим индекс:


```
CREATE INDEX idx_bitmap_region_id ON companies_randomly USING bitmap
(region_id);
ANALYZE companies_randomly;
```

1 запрос:

```
-- 1 запрос
EXPLAIN analyze
select * from companies_randomly
where region_id = '77';

SET optimizer = off;
EXPLAIN analyze
select * from companies_randomly
where region_id = '77';

SET optimizer = ON;
```

Используя GPORCA:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..482.27 rows=25867 width=291) (actual time=0.346..17.933 rows=25876 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..448.47 rows=12934 width=291) (actual time=0.006..10.914 rows=12989 loops=1)
3	Filter: (region_id = 77)
4	Planning time: 13.085 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Pivotal Optimizer (GPORCA)
9	Execution time: 19.338 ms

Используя Postgres Query Optimizer:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=25710 width=291) (actual time=0.471..20.758 rows=25876 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..2262.00 rows=12855 width=291) (actual time=0.014..12.044 rows=12989 loops=1)
3	Filter: (region_id = 77)
4	Planning time: 0.263 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Postgres query optimizer
9	Execution time: 22.130 ms

2 запрос:

```
-- 2 запрос
EXPLAIN analyze
select * from companies_randomly
where region_id = '29';

SET optimizer = off;
EXPLAIN analyze
```

```
select * from companies_randomly
where region_id = '29';
SET optimizer = ON;
```

Используя GPORCA:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..401.09 rows=396 width=291) (actual time=0.863..4.072 rows=472 loops=1)
2	-> Bitmap Heap Scan on companies_randomly (cost=0.00..400.46 rows=198 width=291) (actual time=0.226..2.706 rows=242 loops=1)
3	Recheck Cond: (region_id = 29)
4	-> Bitmap Index Scan on idx_bitmap_region_id (cost=0.00..0.00 rows=0 width=0) (actual time=0.128..0.128 rows=1 loops=1)
5	Index Cond: (region_id = 29)
6	Planning time: 4.172 ms
7	(slice0) Executor memory: 93K bytes.
8	(slice1) Executor memory: 1006K bytes avg x 2 workers, 1006K bytes max (seg0).
9	Memory used: 128000kB
10	Optimizer: Pivotal Optimizer (GPORCA)
11	Execution time: 4.587 ms

Используя Postgres Query Optimizer:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=396 width=291) (actual time=1.710..11.316 rows=472 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..2262.00 rows=198 width=291) (actual time=0.067..9.454 rows=242 loops=1)
3	Filter: (region_id = 29)
4	Planning time: 0.279 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Postgres query optimizer
9	Execution time: 11.832 ms

3 запрос:

```
-- 3 запрос
EXPLAIN analyze
select * from companies_randomly
where region_id = '61';

SET optimizer = off;
EXPLAIN analyze
select * from companies_randomly
where region_id = '61';

SET optimizer = ON;
```

Используя GPORCA:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..412.04 rows=1784 width=291) (actual time=0.804..7.792 rows=1735 loops=1)
2	-> Bitmap Heap Scan on companies_randomly (cost=0.00..409.23 rows=892 width=291) (actual time=0.169..6.213 rows=870 loops=1)
3	Recheck Cond: (region_id = 61)
4	-> Bitmap Index Scan on idx_bitmap_region_id (cost=0.00..0.00 rows=0 width=0) (actual time=0.071..0.071 rows=1 loops=1)
5	Index Cond: (region_id = 61)
6	Planning time: 3.987 ms
7	(slice0) Executor memory: 93K bytes.
8	(slice1) Executor memory: 1006K bytes avg x 2 workers, 1006K bytes max (seg0).
9	Memory used: 128000kB
10	Optimizer: Pivotal Optimizer (GPORCA)
11	Execution time: 8.372 ms

Используя Postgres Query Optimizer:

	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=1784 width=291) (actual time=0.975..10.904 rows=1735 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..2262.00 rows=892 width=291) (actual time=0.027..9.855 rows=870 loops=1)
3	Filter: (region_id = 61)
4	Planning time: 0.345 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Postgres query optimizer
9	Execution time: 11.605 ms

Результат в виде таблицы:

Номер запроса	Оптимизатор	Время планирования	Время выполнения	Использованной памяти	Оценка строк/Реальная цифра	Тип сканирования
1	GPORCA	13.085 ms	19.338 ms	128000 kB	25867/25876	Seq
1	Postgres Query Optimizer	0.263 ms	22.130 ms	128000 kB	25710/25876	Seq
2	GPORCA	4.172 ms	4.587 ms	128000 kB	396/472	Bitmap Index Scan + Bitmap Heap Scan
2	Postgres Query Optimizer	0.279 ms	11.832 ms	128000 kB	396/472	Seq
3	GPORCA	3.987 ms	8.372 ms	128000 kB	1784/1735	Bitmap Index Scan + Bitmap

						Heap Scan
3	Postgres Query Optimizer	0.345 ms	11.605 ms	128000 kB	1784/1735	Seq

Вывод:

Создание Bitmap-индекса по полю `region_id` позволило GPORCA ускорить выполнение запросов для регионов с малым количеством записей: ускорение в 2.2 раза для `region_id = '29'` и на 34% для `region_id = '61'`. Однако для региона '77' (около 26% данных) оптимизатор предпочёл Seq Scan, так как оценка стоимости Index Scan оказалась выше. Сравнение с B-tree индексом показало, что для одноточечных запросов B-tree эффективнее (в 2-5 раз быстрее Bitmap). PostgreSQL Optimizer, как и в пункте 3, не использовал индекс, выполняя Seq Scan для всех запросов.

Удалим индекс:

```
DROP INDEX idx_bitmap_region_id;
```

5. Повторно создайте оба индекса, и выполните некоторые запросы. Определите статистики обращения и использования индексов (используйте материал из документации). Сделайте скриншот результата. Удалите индексы.

Повторное создание индексов:

```
CREATE INDEX idx_btree_region_id ON companies_randomly USING btree (region_id);
CREATE INDEX idx_bitmap_region_id ON companies_randomly USING bitmap (region_id);
```

Обновление статистики:

```
ANALYZE companies_randomly;
```

Выполнение некоторых запросов:

```
SELECT * FROM companies_randomly WHERE region_id = '29';
SELECT * FROM companies_randomly WHERE region_id = '61';
SELECT * FROM companies_randomly WHERE region_id = '77';
```

Для получения статистики обращения был написан следующий скрипт:

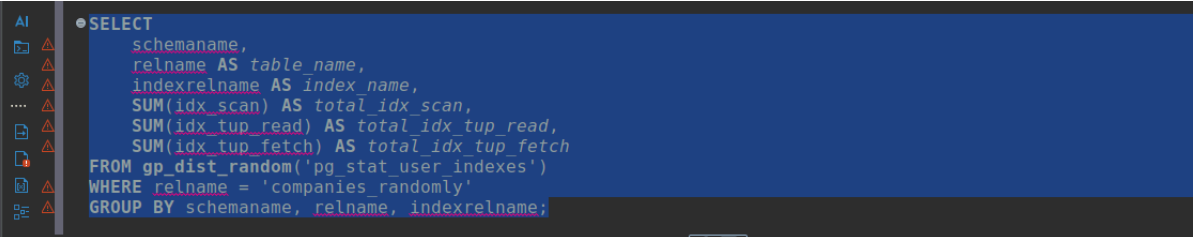
```

SELECT
    schemaname,
    relname AS table_name,
    indexrelname AS index_name,
    SUM(idx_scan) AS total_idx_scan,
    SUM(idx_tup_read) AS total_idx_tup_read,
    SUM(idx_tup_fetch) AS total_idx_tup_fetch
FROM gp_dist_random('pg_stat_user_indexes')
WHERE relname = 'companies_randomly'
GROUP BY schemaname, relname, indexrelname;

```

gp_dist_random - это специальная функция Greenplum, которая позволяет выполнить запрос к системным таблицам/представлениям на всех сегментах одновременно и получить объединенный результат. Без нее мы бы пытались получить данные только с мастера.

Скриншот результата:



Results 1 x

SQL: SELECT schemaname, relname AS table_name, indexrelname AS index_name, SUM(idx_scan) AS total_idx_scan, SUM(idx_tup_read) AS total_idx_tup_read, SUM(idx_tup_fetch) AS total_idx_tup_fetch FROM gp_dist_random('pg_stat_user_indexes') WHERE relname = 'companies_randomly' GROUP BY schemaname, relname, indexrelname;

	schemaname	table_name	index_name	total_idx_scan	total_idx_tup_read	total_idx_tup_fetch
1	public	companies_randomly	idx_bitmap_region_id	0	0	0
2	public	companies_randomly	idx_btree_region_id	28	6,612	6,612

idx_scan - сколько раз индекс был использован при выполнении запросов.

idx_tup_read - сколько записей в индексе было просмотрено.

idx_tup_fetch - сколько реальных строк таблицы было получено по ссылкам из индекса.

Удаление индексов:

```

DROP INDEX idx_btree_region_id;
DROP INDEX idx_bitmap_region_id;

```

6. Постройте любой уникальный индекс для какой-либо из четырех таблиц.

Создание индекса:

```
CREATE UNIQUE INDEX idx_unique_id ON companies_by_id (id);
```

Вставка со значением id которого еще нет в таблице проходит успешно:

```
INSERT INTO companies_by_id VALUES (  
    123,  
    123,  
    123,  
    '000',  
    '000',  
    32,  
    '2026-01-28',  
    NULL,  
    '150007',  
    '150007',  
    'КРАСНОДАРСКИЙ КРАЙ',  
    'Прикубанский округ',  
    'Краснодар',  
    '',  
    'Красная ул',  
    'Д42',  
    'К4',  
    '123',  
    999999,  
    123  
);
```

Statistics 1 x	
Name	Value
Updated Rows	1
Execute time	0.047s
Start time	Tue May 26 11:00:51 MSK 2026
Finish time	Tue May 26 11:00:51 MSK 2026
Query	<pre> INSERT INTO companies_by_id VALUES (123, 123, 123, 'ООО', 'ООО', 32, '2026-01-28', NULL, '150007', '150007', 'КРАСНОДАРСКИЙ КРАЙ', 'Прикубанский округ', 'Краснодар', '', 'Красная ул', 'Д42', 'К4', '123', 999999, 123) </pre>

А если попытаться вставить строку с таким id, который уже есть, то будет ошибка:

```

INSERT INTO companies_by_id VALUES (
    123,
    123,
    123,
    'ООО',
    'ООО',
    32,
    '2026-01-28',
    NULL,
    '150007',
    '150007',
    'КРАСНОДАРСКИЙ КРАЙ',
    'Прикубанский округ',
    'Краснодар',
    '',
    'Красная ул',
    'Д42',
    'К4',

```

```
'123',  
5047675,  
123  
);
```

```
●INSERT INTO companies_by_id VALUES (  
123,  
123,  
123,  
'000',  
'000',  
32,  
'2026-01-28',  
NULL,  
'150007',  
'150007',  
'КРАСНОДАРСКИЙ КРАЙ',  
'Прикубанский округ',  
'Краснодар',  
,  
'Красная ул',  
'Д42',  
'К4',  
'123',  
5047675,  
123  
);
```

Statistics 1 x

SQL Error [23505]: ERROR: duplicate key value violates unique constraint "idx_unique_id" (seg0 172.20.0.3:6000 pid=613)
Detail: Key (id)=(5047675) already exists.
Error position:

7. Разработайте схему списочного секционирования данных. Выберите 4 самых «больших» региона и создайте для них партии, а для записей для остальных регионов предусмотрите партию по умолчанию. Создайте соответствующую таблицу и загрузите в нее данные. Выполните запросы с использованием обоих оптимизаторов, сравните получившиеся планы запросов и метрики, оформите результаты в виде таблицы.

Скрипт для нахождения 4 самых больших региона:

```
SELECT region_id, COUNT(*) as cnt  
FROM companies  
GROUP BY region_id  
ORDER BY COUNT(*) DESC  
LIMIT 4;
```


The screenshot shows a database interface with a SQL query editor at the top and a results grid below. The query is: `SELECT region_id, COUNT(*) as cnt FROM companies GROUP BY region_id ORDER BY COUNT(*) DESC LIMIT 4;`. The results grid shows the top 4 regions by count.

	region_id	cnt
1	77	25,876
2	78	9,033
3	66	3,891
4	50	3,668

Значит самые большие регионы следующие: 77, 78, 66, 50.

Создание таблицы с партиционированием и загрузка данных в нее:

```
CREATE TABLE companies_partition_list (
    LIKE companies
)
DISTRIBUTED BY (region_id)
PARTITION BY LIST (region_id)
(
    PARTITION r77 VALUES ('77'),
    PARTITION r78 VALUES ('78'),
    PARTITION r66 VALUES ('66'),
    PARTITION r50 VALUES ('50'),
    DEFAULT PARTITION r_other
);

INSERT INTO companies_partition_list SELECT * FROM companies;

ANALYZE companies_partition_list;
```

Проверим партиции:

```
SELECT
    tablename,
    partitionname,
    partitiontype,
    partitionlistvalues
FROM pg_partitions
WHERE tablename = 'companies_partition_list';
```

```

SELECT
    tablename,
    partitionname,
    partitiontype,
    partitionlistvalues
FROM pg_partitions
WHERE tablename = 'companies_partition_list';

```

pg_partitions 1 × Output

SELECT tablename, partitionname, partitiontype, partitionlistvalues

	AZ tablename	AZ partitionname	AZ partitiontype	AZ partitionlistvalues
1	companies_partition_list	r77	list	77
2	companies_partition_list	r78	list	78
3	companies_partition_list	r66	list	66
4	companies_partition_list	r50	list	50
5	companies_partition_list	r_other	list	

Можем прочесть данные из партии:

```
SELECT * FROM companies_partition_list_1 prt r77;
```

```

SELECT * FROM companies_partition_list_1 prt r77;

```

companies_partition_list_1 prt_r77 1 × Output

SELECT * FROM companies_partition_list_1 prt r77

	AZ gorod_name	AZ naspunkt_name	AZ street_name	AZ dom	AZ korp	AZ kvart	123 id	123 region_id
1			РАДИЩЕВСКАЯ ВЕРХН. УЛ.	Д.5	СТР.5		6,144,653	77
2			ШОЛОХОВА УЛ.	Д.13		КВ.1А	3,879,451	77
3			ВРАЧЕБНЫЙ ПР-Д	Д.10		ОФ 1	4,077,191	77
4			ЗЕЛЁНЫЙ ПР-КТ	Д.17	СТР.1		5,652,616	77
5			ЛЕНИНСКИЙ ПР-КТ	Д. 29	СТР. 6	Э 1 ПОМ II К 1 ОФ 6	3,955,202	77
6			ВАЛДАЙСКИЙ ПР-Д	Д.8			1,552,497	77
7			ПЯТНИЦКАЯ УЛ.	Д.18	К.3		2,962,934	77
8			МЫТНАЯ УЛ.	Д.54		КВ.7	3,011,698	77
9			ВАВИЛОВА УЛ.	Д. 21	СТР. 20		4,174,302	77
10			ЗАТОННАЯ УЛ.	Д.4		КВ.100	6,176,216	77
11			НИКИТСКАЯ Б. УЛ.	Д.11/4	К.3		7,033,077	77
12			ИЗМАЙЛОВСКОЕ Ш.	Д.71	К.4ГД		4,909,648	77

1 запрос:

```

-- 1 запрос
EXPLAIN analyze
select * from companies_partition_list
where region_id = '77';

SET optimizer = off;
EXPLAIN analyze
select * from companies_partition_list
where region_id = '77';

SET optimizer = ON;

```

Используя GPORCA:

```
-- 1 запрос
EXPLAIN analyze
select * from companies_partition_list
where region_id = '77';
```

Results 1 X

EXPLAIN analyze select * from companies_partition_list Enter a SQL expression to filter results (use Ctrl+Space)

Grid	AZ QUERY PLAN
1	Gather Motion 1:1 (slice1; segments: 1) (cost=0.00..476.30 rows=25876 width=285) (actual time=3.276..63.162 rows=25876 loops=1)
2	-> Sequence (cost=0.00..443.19 rows=12938 width=285) (actual time=1.444..22.222 rows=25876 loops=1)
3	-> Partition Selector for companies_partition_list (dynamic scan id: 1) (cost=10.00..100.00 rows=50 width=4) (never executed)
4	Partitions selected: 1 (out of 5)
5	-> Dynamic Seq Scan on companies_partition_list (dynamic scan id: 1) (cost=0.00..443.19 rows=12938 width=285) (actual time=1.427..19.798 rows=25876 loops=1)
6	Filter: (region_id = 77)
7	Partitions scanned: 1 (out of 5).
8	Planning time: 19.966 ms
9	(slice0) Executor memory: 165K bytes.
10	(slice1) Executor memory: 177K bytes (seg0).
11	Memory used: 128000kB
12	Optimizer: Pivotal Optimizer (GPORCA)
13	Execution time: 65.301 ms

Используя Postgres Query Optimizer:

```
SET optimizer = off;
EXPLAIN analyze
select * from companies_partition_list
where region_id = '77';
```

Results 1 X

EXPLAIN analyze select * from companies_partition_list Enter a SQL expression to filter results (use Ctrl+Space)

Grid	AZ QUERY PLAN
1	Gather Motion 1:1 (slice1; segments: 1) (cost=0.00..1888.60 rows=25877 width=260) (actual time=0.562..29.949 rows=25876 loops=1)
2	-> Append (cost=0.00..1888.60 rows=12939 width=260) (actual time=0.006..16.332 rows=25876 loops=1)
3	-> Seq Scan on companies_partition_list_1_prt_r77 (cost=0.00..561.45 rows=12938 width=260) (actual time=0.006..5.424 rows=25876 loops=1)
4	Filter: (region_id = 77)
5	-> Seq Scan on companies_partition_list_1_prt_r_other (cost=0.00..1327.15 rows=1 width=304) (never executed)
6	Filter: (region_id = 77)
7	Planning time: 22.422 ms
8	(slice0) Executor memory: 166K bytes.
9	(slice1) Executor memory: 145K bytes (seg0).
10	Memory used: 128000kB
11	Optimizer: Postgres query optimizer
12	Execution time: 31.286 ms

2 запрос:

```
-- 2 запрос
EXPLAIN analyze
select * from companies_partition_list
where region_id = '78';

SET optimizer = off;
EXPLAIN analyze
select * from companies_partition_list
where region_id = '78';

SET optimizer = ON;
```

Используя GPORCA:

...	EXPLAIN analyze select * from companies_partition_list where region_id = '78';
Results 1 x	EXPLAIN analyze select * from companies_partition_list Enter a SQL expression to filter results (use Ctrl+Space)
Grid	AZ QUERY PLAN
1	Gather Motion 1:1 (slice1; segments: 1) (cost=0.00..446.82 rows=9033 width=285) (actual time=0.399..12.661 rows=9033 loops=1)
2	-> Sequence (cost=0.00..435.26 rows=4517 width=285) (actual time=0.054..3.851 rows=9033 loops=1)
3	-> Partition Selector for companies_partition_list (dynamic scan id: 1) (cost=10.00..100.00 rows=50 width=4) (never executed)
4	Partitions selected: 1 (out of 5)
5	-> Dynamic Seq Scan on companies_partition_list (dynamic scan id: 1) (cost=0.00..435.26 rows=4517 width=285) (actual time=0.034..3.082 rows=9033 loops=1)
6	Filter: (region_id = 78)
7	Partitions scanned: 1 (out of 5)
8	Planning time: 3.505 ms
9	(slice0) Executor memory: 165K bytes.
10	(slice1) Executor memory: 177K bytes (seg1).
11	Memory used: 128000kB
12	Optimizer: Pivot Optimizer (GPORCA)
13	Execution time: 13.606 ms

Используя Postgres Query Optimizer:

...	SET optimizer = off; EXPLAIN analyze select * from companies_partition_list where region_id = '78';
Results 1 x	EXPLAIN analyze select * from companies_partition_list Enter a SQL expression to filter results (use Ctrl+Space)
Grid	AZ QUERY PLAN
1	Gather Motion 1:1 (slice1; segments: 1) (cost=0.00..1528.06 rows=9034 width=276) (actual time=0.446..17.485 rows=9033 loops=1)
2	-> Append (cost=0.00..1528.06 rows=4517 width=276) (actual time=0.006..12.629 rows=9033 loops=1)
3	-> Seq Scan on companies_partition_list_1_prt_r78 (cost=0.00..200.91 rows=4517 width=276) (actual time=0.006..1.658 rows=9033 loops=1)
4	Filter: (region_id = 78)
5	-> Seq Scan on companies_partition_list_1_prt_r_other (cost=0.00..1327.15 rows=1 width=304) (never executed)
6	Filter: (region_id = 78)
7	Planning time: 8.535 ms
8	(slice0) Executor memory: 166K bytes.
9	(slice1) Executor memory: 145K bytes (seg1).
10	Memory used: 128000kB
11	Optimizer: Postgres query optimizer
12	Execution time: 17.974 ms

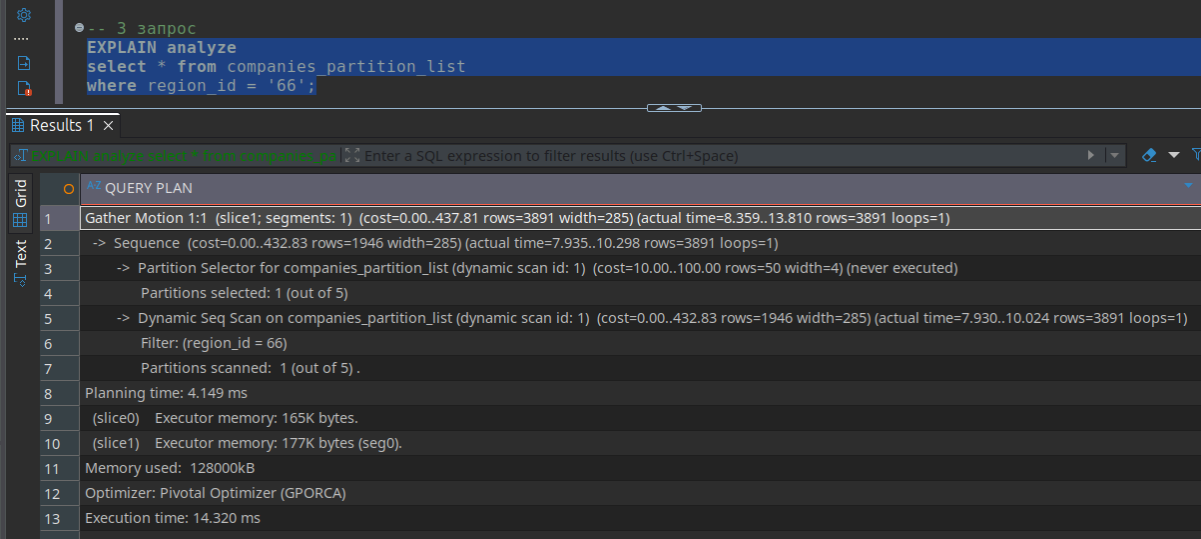
3 запрос:

```
-- 3 запрос
EXPLAIN analyze
select * from companies_partition_list
where region_id = '66';

SET optimizer = off;
EXPLAIN analyze
select * from companies_partition_list
where region_id = '66';

SET optimizer = ON;
```

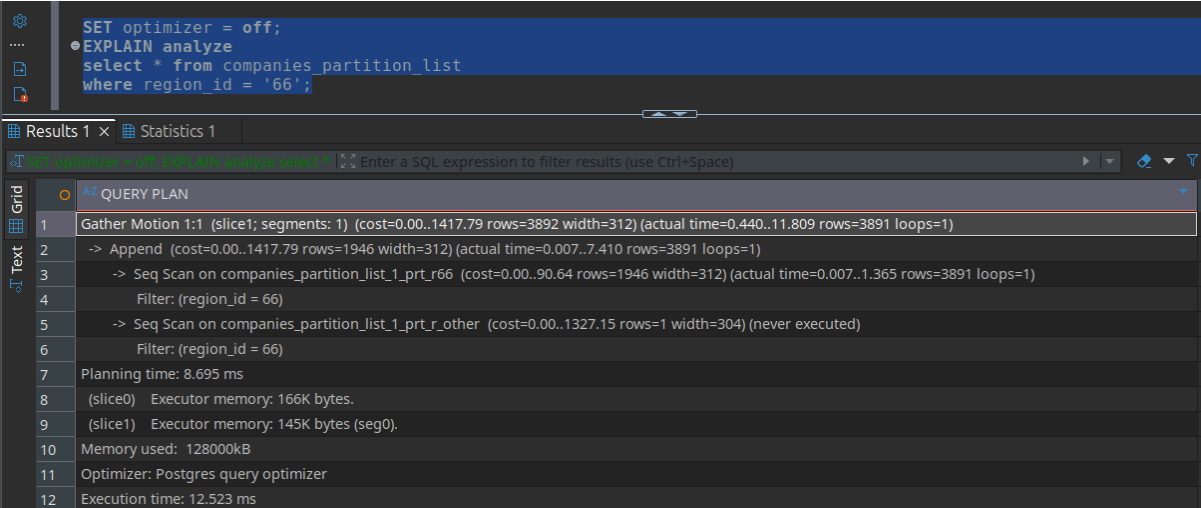
Используя GPORCA:



```
-- 3 запрос
EXPLAIN analyze
select * from companies_partition_list
where region_id = '66';
```

Grid	Text
1	AZ QUERY PLAN
1	Gather Motion 1:1 (slice1; segments: 1) (cost=0.00..437.81 rows=3891 width=285) (actual time=8.359..13.810 rows=3891 loops=1)
2	-> Sequence (cost=0.00..432.83 rows=1946 width=285) (actual time=7.935..10.298 rows=3891 loops=1)
3	-> Partition Selector for companies_partition_list (dynamic scan id: 1) (cost=10.00..100.00 rows=50 width=4) (never executed)
4	Partitions selected: 1 (out of 5)
5	-> Dynamic Seq Scan on companies_partition_list (dynamic scan id: 1) (cost=0.00..432.83 rows=1946 width=285) (actual time=7.930..10.024 rows=3891 loops=1)
6	Filter: (region_id = 66)
7	Partitions scanned: 1 (out of 5) .
8	Planning time: 4.149 ms
9	(slice0) Executor memory: 165K bytes.
10	(slice1) Executor memory: 177K bytes (seg0).
11	Memory used: 128000kB
12	Optimizer: Pivotal Optimizer (GPORCA)
13	Execution time: 14.320 ms

Используя Postgres Query Optimizer:



```
SET optimizer = off;
EXPLAIN analyze
select * from companies_partition_list
where region_id = '66';
```

Grid	Text
1	AZ QUERY PLAN
1	Gather Motion 1:1 (slice1; segments: 1) (cost=0.00..1417.79 rows=3892 width=312) (actual time=0.440..11.809 rows=3891 loops=1)
2	-> Append (cost=0.00..1417.79 rows=1946 width=312) (actual time=0.007..7.410 rows=3891 loops=1)
3	-> Seq Scan on companies_partition_list_1_prt_r66 (cost=0.00..90.64 rows=1946 width=312) (actual time=0.007..1.365 rows=3891 loops=1)
4	Filter: (region_id = 66)
5	-> Seq Scan on companies_partition_list_1_prt_r_other (cost=0.00..1327.15 rows=1 width=304) (never executed)
6	Filter: (region_id = 66)
7	Planning time: 8.695 ms
8	(slice0) Executor memory: 166K bytes.
9	(slice1) Executor memory: 145K bytes (seg0).
10	Memory used: 128000kB
11	Optimizer: Postgres query optimizer
12	Execution time: 12.523 ms

4 запрос:

```
-- 4 запрос
EXPLAIN analyze
select * from companies_partition_list
where region_id = '50';

SET optimizer = off;
EXPLAIN analyze
select * from companies_partition_list
where region_id = '50';

SET optimizer = ON;
```

Используя GPORCA:

<pre> -- 4 запрос EXPLAIN analyze select * from companies_partition_list where region_id = '50'; </pre>																											
Results 1 x																											
<pre> EXPLAIN analyze select * from companies_partition_list Enter a SQL expression to filter results (use Ctrl+Space) </pre>																											
Grid	QUERY PLAN <table> <tr> <td>1</td><td>Gather Motion 1:1 (slice1; segments: 1) (cost=0.00..437.42 rows=3668 width=285) (actual time=8.402..12.885 rows=3668 loops=1)</td></tr> <tr> <td>2</td><td>-> Sequence (cost=0.00..432.73 rows=1834 width=285) (actual time=7.854..9.792 rows=3668 loops=1)</td></tr> <tr> <td>3</td><td>-> Partition Selector for companies_partition_list (dynamic scan id: 1) (cost=10.00..100.00 rows=50 width=4) (never executed)</td></tr> <tr> <td>4</td><td>Partitions selected: 1 (out of 5)</td></tr> <tr> <td>5</td><td>-> Dynamic Seq Scan on companies_partition_list (dynamic scan id: 1) (cost=0.00..432.73 rows=1834 width=285) (actual time=7.851..9.558 rows=3668 loops=1)</td></tr> <tr> <td>6</td><td>Filter: (region_id = 50)</td></tr> <tr> <td>7</td><td>Partitions scanned: 1 (out of 5)</td></tr> <tr> <td>8</td><td>Planning time: 13.647 ms</td></tr> <tr> <td>9</td><td>(slice0) Executor memory: 165K bytes.</td></tr> <tr> <td>10</td><td>(slice1) Executor memory: 177K bytes (seg1).</td></tr> <tr> <td>11</td><td>Memory used: 128000kB</td></tr> <tr> <td>12</td><td>Optimizer: Pivotal Optimizer (GPORCA)</td></tr> <tr> <td>13</td><td>Execution time: 13.343 ms</td></tr> </table>	1	Gather Motion 1:1 (slice1; segments: 1) (cost=0.00..437.42 rows=3668 width=285) (actual time=8.402..12.885 rows=3668 loops=1)	2	-> Sequence (cost=0.00..432.73 rows=1834 width=285) (actual time=7.854..9.792 rows=3668 loops=1)	3	-> Partition Selector for companies_partition_list (dynamic scan id: 1) (cost=10.00..100.00 rows=50 width=4) (never executed)	4	Partitions selected: 1 (out of 5)	5	-> Dynamic Seq Scan on companies_partition_list (dynamic scan id: 1) (cost=0.00..432.73 rows=1834 width=285) (actual time=7.851..9.558 rows=3668 loops=1)	6	Filter: (region_id = 50)	7	Partitions scanned: 1 (out of 5)	8	Planning time: 13.647 ms	9	(slice0) Executor memory: 165K bytes.	10	(slice1) Executor memory: 177K bytes (seg1).	11	Memory used: 128000kB	12	Optimizer: Pivotal Optimizer (GPORCA)	13	Execution time: 13.343 ms
1	Gather Motion 1:1 (slice1; segments: 1) (cost=0.00..437.42 rows=3668 width=285) (actual time=8.402..12.885 rows=3668 loops=1)																										
2	-> Sequence (cost=0.00..432.73 rows=1834 width=285) (actual time=7.854..9.792 rows=3668 loops=1)																										
3	-> Partition Selector for companies_partition_list (dynamic scan id: 1) (cost=10.00..100.00 rows=50 width=4) (never executed)																										
4	Partitions selected: 1 (out of 5)																										
5	-> Dynamic Seq Scan on companies_partition_list (dynamic scan id: 1) (cost=0.00..432.73 rows=1834 width=285) (actual time=7.851..9.558 rows=3668 loops=1)																										
6	Filter: (region_id = 50)																										
7	Partitions scanned: 1 (out of 5)																										
8	Planning time: 13.647 ms																										
9	(slice0) Executor memory: 165K bytes.																										
10	(slice1) Executor memory: 177K bytes (seg1).																										
11	Memory used: 128000kB																										
12	Optimizer: Pivotal Optimizer (GPORCA)																										
13	Execution time: 13.343 ms																										

Используя Postgres Query Optimizer:

<pre> SET optimizer = off; EXPLAIN analyze select * from companies_partition_list where region_id = '50'; </pre>																									
Results 1 x Statistics 1																									
<pre> SET optimizer = off, EXPLAIN analyze select * Enter a SQL expression to filter results (use Ctrl+Space) </pre>																									
Grid	QUERY PLAN <table> <tr> <td>1</td><td>Gather Motion 1:1 (slice1; segments: 1) (cost=0.00..1412.00 rows=3669 width=300) (actual time=0.302..12.348 rows=3668 loops=1)</td></tr> <tr> <td>2</td><td>-> Append (cost=0.00..1412.00 rows=1835 width=300) (actual time=0.008..9.207 rows=3668 loops=1)</td></tr> <tr> <td>3</td><td>-> Seq Scan on companies_partition_list_1_prt_r50 (cost=0.00..84.85 rows=1834 width=300) (actual time=0.008..0.926 rows=3668 loops=1)</td></tr> <tr> <td>4</td><td>Filter: (region_id = 50)</td></tr> <tr> <td>5</td><td>-> Seq Scan on companies_partition_list_1_prt_r_other (cost=0.00..1327.15 rows=1 width=304) (never executed)</td></tr> <tr> <td>6</td><td>Filter: (region_id = 50)</td></tr> <tr> <td>7</td><td>Planning time: 8.200 ms</td></tr> <tr> <td>8</td><td>(slice0) Executor memory: 166K bytes.</td></tr> <tr> <td>9</td><td>(slice1) Executor memory: 145K bytes (seg1).</td></tr> <tr> <td>10</td><td>Memory used: 128000kB</td></tr> <tr> <td>11</td><td>Optimizer: Postgres query optimizer</td></tr> <tr> <td>12</td><td>Execution time: 13.078 ms</td></tr> </table>	1	Gather Motion 1:1 (slice1; segments: 1) (cost=0.00..1412.00 rows=3669 width=300) (actual time=0.302..12.348 rows=3668 loops=1)	2	-> Append (cost=0.00..1412.00 rows=1835 width=300) (actual time=0.008..9.207 rows=3668 loops=1)	3	-> Seq Scan on companies_partition_list_1_prt_r50 (cost=0.00..84.85 rows=1834 width=300) (actual time=0.008..0.926 rows=3668 loops=1)	4	Filter: (region_id = 50)	5	-> Seq Scan on companies_partition_list_1_prt_r_other (cost=0.00..1327.15 rows=1 width=304) (never executed)	6	Filter: (region_id = 50)	7	Planning time: 8.200 ms	8	(slice0) Executor memory: 166K bytes.	9	(slice1) Executor memory: 145K bytes (seg1).	10	Memory used: 128000kB	11	Optimizer: Postgres query optimizer	12	Execution time: 13.078 ms
1	Gather Motion 1:1 (slice1; segments: 1) (cost=0.00..1412.00 rows=3669 width=300) (actual time=0.302..12.348 rows=3668 loops=1)																								
2	-> Append (cost=0.00..1412.00 rows=1835 width=300) (actual time=0.008..9.207 rows=3668 loops=1)																								
3	-> Seq Scan on companies_partition_list_1_prt_r50 (cost=0.00..84.85 rows=1834 width=300) (actual time=0.008..0.926 rows=3668 loops=1)																								
4	Filter: (region_id = 50)																								
5	-> Seq Scan on companies_partition_list_1_prt_r_other (cost=0.00..1327.15 rows=1 width=304) (never executed)																								
6	Filter: (region_id = 50)																								
7	Planning time: 8.200 ms																								
8	(slice0) Executor memory: 166K bytes.																								
9	(slice1) Executor memory: 145K bytes (seg1).																								
10	Memory used: 128000kB																								
11	Optimizer: Postgres query optimizer																								
12	Execution time: 13.078 ms																								

5 запрос:

```

EXPLAIN analyze
SELECT DISTINCT region_id
FROM companies_partition_list
WHERE region_id NOT IN ('77', '78', '66', '50')
ORDER BY region_id;

```

Используя GPORCA:

1 (77)	GPORCA	19.966 ms	65.301 ms	25876/25876
1 (77)	Postgres Query Optimizer	22.422 ms	31.286 ms	25877/25876
2 (78)	GPORCA	3.505 ms	13.606 ms	9033/9033
2 (78)	Postgres Query Optimizer	8.535 ms	17.974 ms	9034/9033
3 (66)	GPORCA	4.149 ms	14.320 ms	3891/3891
3 (66)	Postgres Query Optimizer	8.695 ms	12.523 ms	3892/3891
4 (50)	GPORCA	13.647 ms	13.343 ms	3668/3668
4 (50)	Postgres Query Optimizer	8.200 ms	13.078 ms	3669/3668
5 other	GPORCA	21.167 ms	30.687 ms	88/88
5 other	Postgres Query Optimizer	8.277 ms	10.751 ms	88/88

Также сравним с обычными запросами к companies_randomly:
1 запрос:

```
-- 1 запрос
SET optimizer = ON;
SET optimizer = OFF;
EXPLAIN analyze
select * from companies_randomly
where region_id = '77';
```

GPORCA:

EXPLAIN analyze
select * from companies_randomly
where region_id = '77';

Results 1 x

EXPLAIN analyze select * from companies_randomly where region_id = '77'; Enter a SQL expression to filter results (use Ctrl+Space)

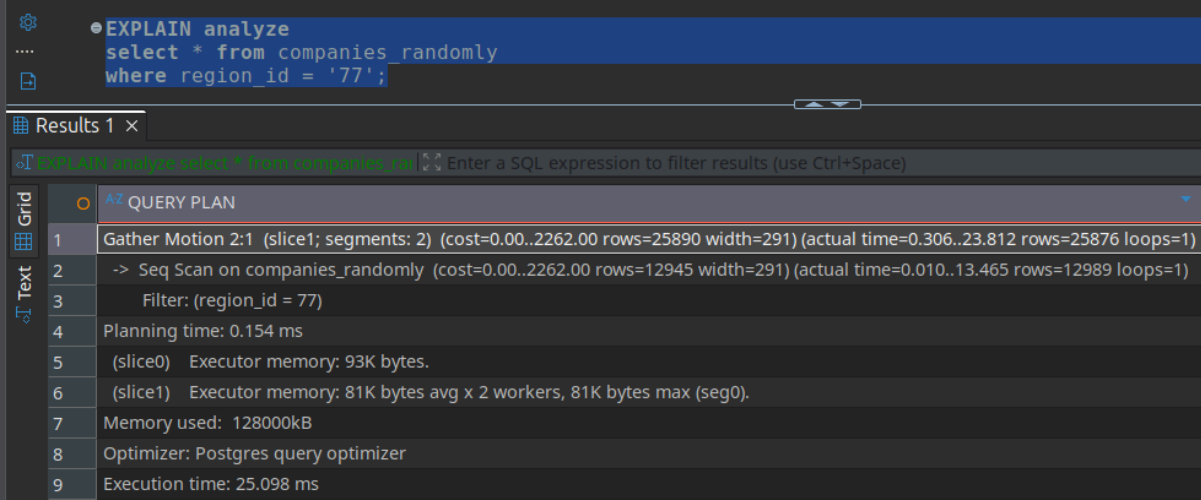
Grid

AZ QUERY PLAN

1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..482.31 rows=25890 width=291) (actual time=0.765..25.826 rows=25876 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..448.48 rows=12945 width=291) (actual time=0.024..13.382 rows=12989 loops=1)
3	Filter: (region_id = 77)
4	Planning time: 5.968 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Pivotal Optimizer (GPORCA)
9	Execution time: 27.266 ms

Text

Postgres Query Optimizer:



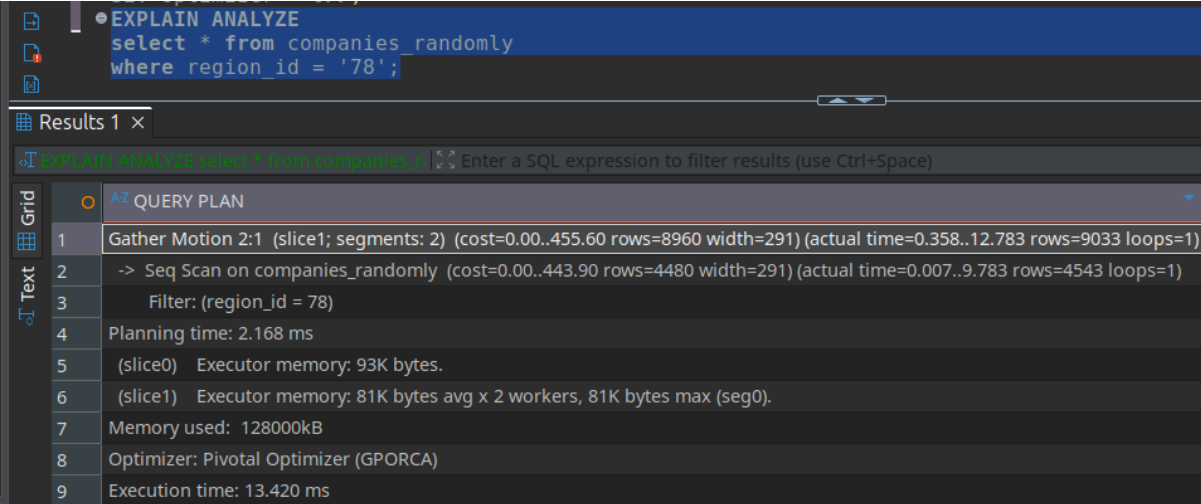
The screenshot shows a PostgreSQL query execution plan for the query: `EXPLAIN analyze select * from companies_randomly where region_id = '77';`. The plan is displayed in a table with 9 rows. The first row is the title 'Az QUERY PLAN'. The second row is 'Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=25890 width=291) (actual time=0.306..23.812 rows=25876 loops=1)'. The third row is '-> Seq Scan on companies_randomly (cost=0.00..2262.00 rows=12945 width=291) (actual time=0.010..13.465 rows=12989 loops=1)'. The fourth row is 'Filter: (region_id = 77)'. The fifth row is 'Planning time: 0.154 ms'. The sixth row is '(slice0) Executor memory: 93K bytes.'. The seventh row is '(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0)'. The eighth row is 'Memory used: 128000kB'. The ninth row is 'Optimizer: Postgres query optimizer'. The tenth row is 'Execution time: 25.098 ms'.

Grid	Text
1	Az QUERY PLAN
2	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=25890 width=291) (actual time=0.306..23.812 rows=25876 loops=1)
3	-> Seq Scan on companies_randomly (cost=0.00..2262.00 rows=12945 width=291) (actual time=0.010..13.465 rows=12989 loops=1)
4	Filter: (region_id = 77)
5	Planning time: 0.154 ms
6	(slice0) Executor memory: 93K bytes.
7	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
8	Memory used: 128000kB
9	Optimizer: Postgres query optimizer
10	Execution time: 25.098 ms

2 запрос:

```
-- 2 запрос
SET optimizer = ON;
SET optimizer = OFF;
EXPLAIN ANALYZE
select * from companies_randomly
where region_id = '78';
```

GPORCA:



The screenshot shows a PostgreSQL query execution plan for the query: `EXPLAIN ANALYZE select * from companies_randomly where region_id = '78';`. The plan is displayed in a table with 9 rows. The first row is the title 'Az QUERY PLAN'. The second row is 'Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..455.60 rows=8960 width=291) (actual time=0.358..12.783 rows=9033 loops=1)'. The third row is '-> Seq Scan on companies_randomly (cost=0.00..443.90 rows=4480 width=291) (actual time=0.007..9.783 rows=4543 loops=1)'. The fourth row is 'Filter: (region_id = 78)'. The fifth row is 'Planning time: 2.168 ms'. The sixth row is '(slice0) Executor memory: 93K bytes.'. The seventh row is '(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0)'. The eighth row is 'Memory used: 128000kB'. The ninth row is 'Optimizer: Pivotal Optimizer (GPORCA)'. The tenth row is 'Execution time: 13.420 ms'.

Grid	Text
1	Az QUERY PLAN
2	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..455.60 rows=8960 width=291) (actual time=0.358..12.783 rows=9033 loops=1)
3	-> Seq Scan on companies_randomly (cost=0.00..443.90 rows=4480 width=291) (actual time=0.007..9.783 rows=4543 loops=1)
4	Filter: (region_id = 78)
5	Planning time: 2.168 ms
6	(slice0) Executor memory: 93K bytes.
7	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
8	Memory used: 128000kB
9	Optimizer: Pivotal Optimizer (GPORCA)
10	Execution time: 13.420 ms

Postgres Query Optimizer:

```
SET optimizer = OFF;
EXPLAIN ANALYZE
select * from companies_randomly
where region_id = '78';
```

Results 1 x Statistics 1

EXPLAIN ANALYZE select * from companies_randomly where region_id = '78'; Enter a SQL expression to filter results (use Ctrl+Space)

Grid	Text
1	AZ QUERY PLAN
2	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=8960 width=291) (actual time=0.382..15.202 rows=9033 loops=1)
3	-> Seq Scan on companies_randomly (cost=0.00..2262.00 rows=4480 width=291) (actual time=0.008..11.112 rows=4543 loops=1)
4	Filter: (region_id = 78)
5	Planning time: 0.146 ms
6	(slice0) Executor memory: 93K bytes.
7	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
8	Memory used: 128000kB
9	Optimizer: Postgres query optimizer
10	Execution time: 15.909 ms

3 запрос:

```
-- 3 запрос
SET optimizer = ON;
SET optimizer = OFF;
EXPLAIN ANALYZE
select * from companies_randomly
where region_id = '66';
```

GPORCA:

```
EXPLAIN ANALYZE
select * from companies_randomly
where region_id = '66';
```

Results 1 x

EXPLAIN ANALYZE select * from companies_randomly where region_id = '66'; Enter a SQL expression to filter results (use Ctrl+Space)

Grid	Text
1	AZ QUERY PLAN
2	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..447.66 rows=3924 width=291) (actual time=0.549..12.119 rows=3891 loops=1)
3	-> Seq Scan on companies_randomly (cost=0.00..442.53 rows=1962 width=291) (actual time=0.014..9.999 rows=1967 loops=1)
4	Filter: (region_id = 66)
5	Planning time: 5.586 ms
6	(slice0) Executor memory: 93K bytes.
7	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
8	Memory used: 128000kB
9	Optimizer: Pivotal Optimizer (GPORCA)
10	Execution time: 12.767 ms

Postgres Query Optimizer:

<pre> EXPLAIN ANALYZE select * from companies_randomly where region_id = '66'; </pre>	
Results 1 x	
EXPLAIN ANALYZE select * from companies_randomly Enter a SQL expression to filter results (use Ctrl+Space)	
Grid	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=3924 width=291) (actual time=0.661..16.130 rows=3891 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..2262.00 rows=1962 width=291) (actual time=0.016..13.762 rows=1967 loops=1)
3	Filter: (region_id = 66)
4	Planning time: 0.313 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Postgres query optimizer
9	Execution time: 17.017 ms

4 запрос:

```

-- 4 запрос
SET optimizer = ON;
SET optimizer = OFF;
EXPLAIN analyze
select * from companies_randomly
where region_id = '50';

```

GPORCA:

<pre> EXPLAIN analyze select * from companies_randomly where region_id = '50'; </pre>	
Results 1 x	
EXPLAIN analyze select * from companies_randomly Enter a SQL expression to filter results (use Ctrl+Space)	
Grid	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..447.23 rows=3650 width=291) (actual time=0.745..16.008 rows=3668 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..442.46 rows=1825 width=291) (actual time=0.042..13.622 rows=1834 loops=1)
3	Filter: (region_id = 50)
4	Planning time: 3.722 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Pivotal Optimizer (GPORCA)
9	Execution time: 16.724 ms

Postgres Query Optimizer:

AI	SET optimizer = OFF; EXPLAIN analyze select * from companies_randomly where region_id = '50';
Results 1 ×	Statistics 1
SET optimizer = OFF; EXPLAIN analyze select * from companies_randomly where region_id = '50'; Enter a SQL expression to filter results (use Ctrl+Space)	
Grid	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=3650 width=291) (actual time=0.959..19.012 rows=3668 loops=1)
2	-> Seq Scan on companies_randomly (cost=0.00..2262.00 rows=1825 width=291) (actual time=0.067..16.509 rows=1834 loops=1)
3	Filter: (region_id = 50)
4	Planning time: 0.316 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Postgres query optimizer
9	Execution time: 19.721 ms

5 запрос:

```
-- 5 запрос
SET optimizer = ON;
SET optimizer = OFF;
EXPLAIN analyze
SELECT DISTINCT region_id
FROM companies_randomly
WHERE region_id NOT IN ('77', '78', '66', '50')
ORDER BY region_id;
```

GPORCA:

<pre> EXPLAIN analyze SELECT DISTINCT region_id FROM companies_randomly WHERE region_id NOT IN ('77', '78', '66', '50') ORDER BY region_id; </pre>	
Results 1 x	
EXPLAIN analyze SELECT DISTINCT region_id Enter a SQL expression to filter results (use Ctrl+Space)	
Grid	AZ QUERY PLAN
Text	1 Gather Motion 2:1 (slice2; segments: 2) (cost=0.00..445.19 rows=87 width=4) (actual time=12.901..12.919 rows=88 loops=1)
	2 Merge Key: region_id
	3 -> GroupAggregate (cost=0.00..445.19 rows=44 width=4) (actual time=12.720..12.731 rows=46 loops=1)
	4 Group Key: region_id
	5 -> Sort (cost=0.00..445.19 rows=44 width=4) (actual time=12.715..12.717 rows=92 loops=1)
	6 Sort Key: region_id
	7 Sort Method: quicksort Memory: 66kB
	8 -> Redistribute Motion 2:2 (slice1; segments: 2) (cost=0.00..445.19 rows=44 width=4) (actual time=12.436..12.688 rows=92 loops=1)
	9 Hash Key: region_id
	10 -> HashAggregate (cost=0.00..445.19 rows=44 width=4) (actual time=12.161..12.179 rows=88 loops=1)
	11 Group Key: region_id
	12 Extra Text: (seg0) Hash chain length 2.8 avg, 6 max, using 31 of 32 buckets; total 0 expansions.
	13
	14 -> Seq Scan on companies_randomly (cost=0.00..441.69 rows=28789 width=4) (actual time=0.008..10.191 rows=28988 loops=1)
	15 Filter: (region_id <> ALL ('77,78,66,50')::integer[])
Record	16 Planning time: 4.302 ms
	17 (slice0) Executor memory: 71K bytes.
	18 (slice1) Executor memory: 148K bytes avg x 2 workers, 148K bytes max (seg0).
	19 (slice2) Executor memory: 64K bytes avg x 2 workers, 64K bytes max (seg0). Work_mem: 33K bytes max.
	20 Memory used: 128000kB
	21 Optimizer: Pivotal Optimizer (GPORCA)
	22 Execution time: 16.982 ms

Postgres Query Optimizer:

```

SET optimizer = OFF;
EXPLAIN analyze
SELECT DISTINCT region_id
FROM companies_randomly
WHERE region_id NOT IN ('77', '78', '66', '50')
ORDER BY region_id;

```

Results 1 x Statistics 1

SET optimizer = OFF; EXPLAIN analyze SELECT | Enter a SQL expression to filter results (use Ctrl+Space)

AZ QUERY PLAN

- Gather Motion 2:1 (slice2; segments: 2) (cost=2659.71..2659.84 rows=53 width=4) (actual time=17.314..17.330 rows=88 loops=1)
- Merge Key: companies_randomly.region_id
- > Sort (cost=2659.71..2659.84 rows=27 width=4) (actual time=17.044..17.048 rows=46 loops=1)
- Sort Key: companies_randomly.region_id
- Sort Method: quicksort Memory: 66kB
- > HashAggregate (cost=2657.66..2658.19 rows=27 width=4) (actual time=17.007..17.021 rows=46 loops=1)
- Group Key: companies_randomly.region_id
- Extra Text: (seg0) Hash chain length 1.8 avg, 3 max, using 25 of 32 buckets; total 0 expansions.
- > Redistribute Motion 2:2 (slice1; segments: 2) (cost=2655.94..2657.00 rows=27 width=4) (actual time=15.412..16.982 rows=92 loops=1)
- Hash Key: companies_randomly.region_id
- > HashAggregate (cost=2655.94..2655.94 rows=27 width=4) (actual time=16.460..16.472 rows=88 loops=1)
- Group Key: companies_randomly.region_id
- Extra Text: (seg0) Hash chain length 2.8 avg, 6 max, using 31 of 32 buckets; total 0 expansions.
- > Seq Scan on companies_randomly (cost=0.00..2512.00 rows=28789 width=4) (actual time=0.011..12.174 rows=28988 loops=1)
- Filter: (region_id <> ALL ('77,78,66,50')::integer[])
- Planning time: 0.195 ms
- (slice0) Executor memory: 71K bytes.
- (slice1) Executor memory: 148K bytes avg x 2 workers, 148K bytes max (seg0).
- (slice2) Executor memory: 152K bytes avg x 2 workers, 152K bytes max (seg0). Work_mem: 33K bytes max.
- Memory used: 128000kB
- Optimizer: Postgres query optimizer
- Execution time: 23.309 ms

Результат в виде таблицы:

Номер запроса	Оптимизатор	Время планирования	Время выполнения
1 (77)	GPORCA	5.968 ms	27.266 ms
1 (77)	Postgres Query Optimizer	0.154 ms	25.098 ms
2 (78)	GPORCA	2.168 ms	13.420 ms
2 (78)	Postgres Query Optimizer	0.146 ms	15.909 ms
3 (66)	GPORCA	5.586 ms	12.767 ms
3 (66)	Postgres Query Optimizer	0.313 ms	17.017 ms
4 (50)	GPORCA	3.722 ms	16.724 ms
4 (50)	Postgres Query Optimizer	0.316 ms	19.721 ms

5 other	GPORCA	4.302 ms	16.982 ms
5 other	Postgres Query Optimizer	0.195 ms	23.309 ms

Вывод:

- Партиционирование не дало однозначного ускорения. Для регионов 77 и 78 запросы к партицированной таблице выполняются медленнее, а время планирования дольше. Для других регионов также тяжело определить есть ли положительный эффект от партицирования, ведь GPORCA замедляет запросы все равно, а Postgres Query Optimizer ускоряет.
- Postgres Query Optimizer работает стабильнее. Он показал ускорение для 3 из 5 запросов по сравнению с непартицированной таблицей
- GPORCA проигрывает на партицированной таблице. GPORCA показал худшие результаты на партицированной таблице для всех запросов, кроме региона 50
- Точность оценки строк у GPORCA идеальная, а Postgres Query Optimizer близко к идеальному.
- Время планирования у Postgres query optimizer значительно меньше

Партиционирование не дало ожидаемого ускорения, потому что таблица имеет относительно небольшой объём данных при котором накладные расходы на проверку партиций и более сложное планирование перевешивают выгоду от Partition Pruning. Postgres Query Optimizer оказался стабильнее на партиционированной таблице, так как его более простая модель оптимизации лучше справляется с небольшими объёмами данных, тогда как GPORCA разработанный для сложных аналитических запросов на гигантских таблицах теряет эффективность на малых данных из-за более высоких затрат на планирование и избыточной сложности оценок. Таким образом, партиционирование в Greenplum оправдано только на очень больших таблицах, где отсечение

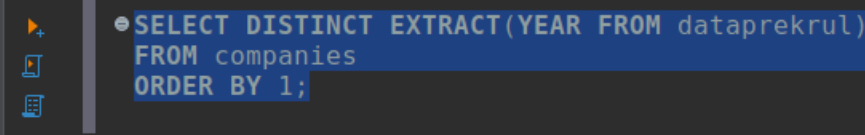
партиций позволяет сократить сканирование на порядки, а на малых объёмах данных параллельное Seq Scan непартицированной таблицы работает не медленнее или даже быстрее.

8. Разработайте схему секционирования (поле dataprekrul). Каждая партиция должна содержать записи за год; записи с наиболее «свежими» записями должны быть помещены в партицию по умолчанию. Создайте соответствующую таблицу и загрузите в нее данные. Выполните запросы п.2 с использованием обоих оптимизаторов, сравните получившиеся планы запросов и метрики, оформите результаты в виде таблицы.

Для создания партицированной таблицы по полю dataprekrul нужно проанализировать какие вообще года там есть:

```
SELECT DISTINCT EXTRACT(YEAR FROM dataprekrul)
FROM companies
ORDER BY 1;
```

Вывод:



SQL Query:

```
SELECT DISTINCT EXTRACT(YEAR FROM dataprekrul)
FROM companies
ORDER BY 1;
```

Results 1 ×

SELECT DISTINCT EXTRACT(YEAR FROM dataprekrul) FROM companies ORDER BY 1;

	123 date_part
1	1,899
2	2,002
3	2,003
4	2,004
5	2,005
6	2,006
7	2,007
8	2,008
9	2,009
10	2,010
11	2,011
12	2,012
13	2,013
14	2,014
15	2,015
16	2,016
17	2,017
18	2,018
19	2,019
20	2,020
21	2,021
22	2,022

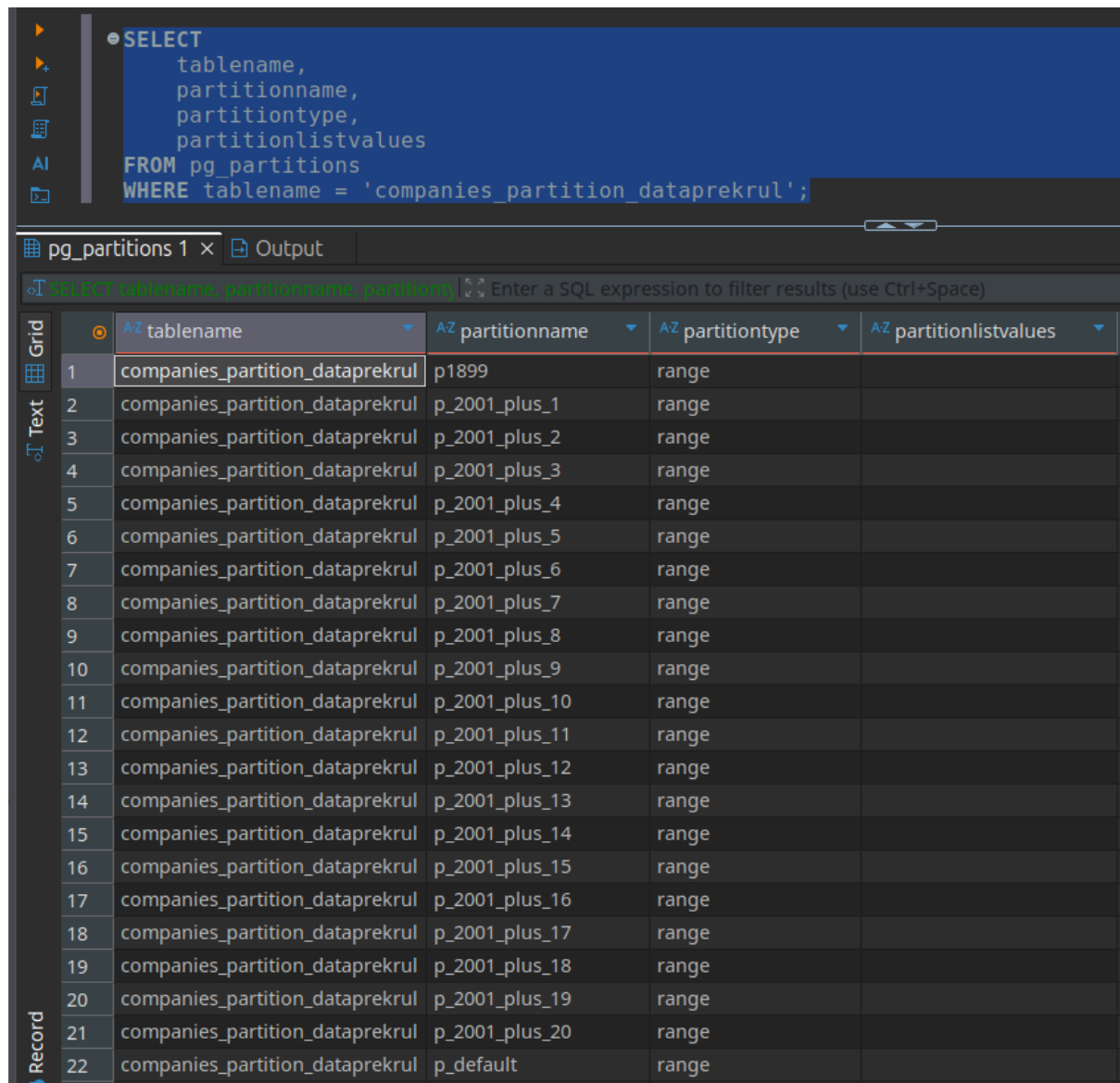
Скрипт для создания партицированной таблицы:

```
CREATE TABLE companies_partition_dataprekrul (
    LIKE companies
)
DISTRIBUTED BY (region_id)
PARTITION BY RANGE (dataprekrul)
(
    PARTITION p1899 START ('1899-01-01') END ('1900-01-01'),
    PARTITION p_2001_plus START ('2002-01-01') END ('2022-01-01')
    EVERY (INTERVAL '1 year'),
    DEFAULT PARTITION p_default
);

INSERT INTO companies_partition_dataprekrul SELECT * FROM companies;

ANALYZE companies_partition_dataprekrul;
```

Посмотрим какие партии созданы:



The screenshot shows a database client interface. At the top, a SQL query is entered in a text editor:

```
SELECT
    tablename,
    partitionname,
    partitiontype,
    partitionlistvalues
FROM pg_partitions
WHERE tablename = 'companies_partition_dataprekrul';
```

Below the query editor, the results are displayed in a table grid. The table has four columns: **tablename**, **partitionname**, **partitiontype**, and **partitionlistvalues**. The results show 22 rows of data, all with the same tablename and partitiontype, but different partitionnames and list values.

	tablename	partitionname	partitiontype	partitionlistvalues
1	companies_partition_dataprekrul	p1899	range	
2	companies_partition_dataprekrul	p_2001_plus_1	range	
3	companies_partition_dataprekrul	p_2001_plus_2	range	
4	companies_partition_dataprekrul	p_2001_plus_3	range	
5	companies_partition_dataprekrul	p_2001_plus_4	range	
6	companies_partition_dataprekrul	p_2001_plus_5	range	
7	companies_partition_dataprekrul	p_2001_plus_6	range	
8	companies_partition_dataprekrul	p_2001_plus_7	range	
9	companies_partition_dataprekrul	p_2001_plus_8	range	
10	companies_partition_dataprekrul	p_2001_plus_9	range	
11	companies_partition_dataprekrul	p_2001_plus_10	range	
12	companies_partition_dataprekrul	p_2001_plus_11	range	
13	companies_partition_dataprekrul	p_2001_plus_12	range	
14	companies_partition_dataprekrul	p_2001_plus_13	range	
15	companies_partition_dataprekrul	p_2001_plus_14	range	
16	companies_partition_dataprekrul	p_2001_plus_15	range	
17	companies_partition_dataprekrul	p_2001_plus_16	range	
18	companies_partition_dataprekrul	p_2001_plus_17	range	
19	companies_partition_dataprekrul	p_2001_plus_18	range	
20	companies_partition_dataprekrul	p_2001_plus_19	range	
21	companies_partition_dataprekrul	p_2001_plus_20	range	
22	companies_partition_dataprekrul	p_default	range	

```
SELECT
    partitiontablename,
    partitionname
FROM pg_partitions
WHERE tablename = 'companies_partition_dataprekrul'
ORDER BY partitionrangestart;
```


SQL Query: `SELECT dataprekrul FROM companies_partition_dataprekrul_1 prt p 2001 plus 17;`

companies_partition_dataprekrul_1 prt p 2001 plus 17 1 × Output

SQL Filter: `SELECT dataprekrul FROM companies_partition_dataprekrul_1 prt p 2001 plus 17;` Enter a SQL expression to filter results (use Ctrl+Space)

	dataprekrul
1	2018-03-26
2	2018-01-09
3	2018-04-02
4	2018-05-07
5	2018-05-14
6	2018-08-27
7	2018-01-29
8	2018-02-19
9	2018-12-11
10	2018-12-07
11	2018-10-01
12	2018-12-24
13	2018-08-08
14	2018-08-28
15	2018-03-20
16	2018-02-19
17	2018-06-18
18	2018-04-28
19	2018-03-12
20	2018-01-29

SQL Query: `SELECT dataprekrul FROM companies_partition_dataprekrul_1 prt p 2001 plus 6;`

companies_partition_dataprekrul_1 prt p 2001 plus 6 1 × Output

SQL Filter: `SELECT dataprekrul FROM companies_partition_dataprekrul_1 prt p 2001 plus 6;` Enter a SQL expression to filter results (use Ctrl+Space)

	dataprekrul
1	2007-12-01
2	2007-03-03
3	2007-10-30
4	2007-12-24
5	2007-12-08
6	2007-02-19
7	2007-10-06
8	2007-01-12
9	2007-09-10
10	2007-09-22
11	2007-01-11
12	2007-02-23
13	2007-08-18
14	2007-05-23
15	2007-03-03
16	2007-04-16
17	2007-10-12
18	2007-10-18
19	2007-02-23
20	2007-02-23
21	2007-01-29
22	2007-04-16

Как мы видим партиции создались верно.

В задании требуется выполнить запросы как в пункте 2, но так как в п.2 запросы где фильтр по region_id, а в данном пункте у нас партиции по dataprekrul, то это не несет в себе смысла и партиции будут всегда читаться все.

Например запрос:

```
EXPLAIN ANALYZE
SELECT * FROM companies_partition_dataprekrul
WHERE region_id = '77';
```

Обрабатывает все партиции:

Grid	1	Gather Motion 1:1 (slice1; segments: 1) (cost=0.00..481.11 rows=25876 width=284) (actual time=0.403..27.793 rows=25876 loops=1)
Text	2	-> Sequence (cost=0.00..448.11 rows=12938 width=284) (actual time=0.027..14.732 rows=25876 loops=1)
	3	-> Partition Selector for companies_partition_dataprekrul (dynamic scan id: 1) (cost=10.00..100.00 rows=50 width=4) (never executed)
	4	Partitions selected: 22 (out of 22)
	5	-> Dynamic Seq Scan on companies_partition_dataprekrul (dynamic scan id: 1) (cost=0.00..448.11 rows=12938 width=284) (actual time=0.023..13.243 rows=25876 loops=1)
	6	Filter: (region_id = 77)
	7	Partitions scanned: 22 (out of 22).
Record	8	Planning time: 6.398 ms
	9	(slice0) Executor memory: 165K bytes.
	10	(slice1) Executor memory: 561K bytes (seg0).
	11	Memory used: 128000kB
	12	Optimizer: Pivotal Optimizer (GPORCA)
	13	Execution time: 29.047 ms

Поэтому рассмотрим запросы где будет отсечение партиций.

Запросы 1:

```
-- 1 запросы
EXPLAIN ANALYZE
SELECT * FROM companies_partition_dataprekrul
WHERE dataprekrul = '2021-01-15';
EXPLAIN ANALYZE
SELECT * FROM companies_by_region
WHERE dataprekrul = '2021-01-15';
```

companies_partition_dataprekrul:

GPORCA:

AI	1 запросы
📄	EXPLAIN ANALYZE SELECT * FROM companies_partition_dataprekru WHERE dataprekru = '2021-01-15';
⚙️	
Results 1 ×	Output
🔍	EXPLAIN ANALYZE SELECT * FROM companies; Enter a SQL expression to filter results (use Ctrl+Space)
Grid	QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..431.03 rows=16 width=284) (actual time=0.995..0.999 rows=16 loops=1)
2	-> Sequence (cost=0.00..431.01 rows=8 width=284) (actual time=0.098..0.419 rows=11 loops=1)
3	-> Partition Selector for companies_partition_dataprekru (dynamic scan id: 1) (cost=10.00..100.00 rows=50 width=4) (never executed)
4	Partitions selected: 1 (out of 22)
5	-> Dynamic Seq Scan on companies_partition_dataprekru (dynamic scan id: 1) (cost=0.00..431.01 rows=8 width=284) (actual time=0.091..0.408 rows=11 loops=1)
6	Filter: (dataprekru = '2021-01-15'::date)
7	Partitions scanned: Avg 1.0 (out of 22) x 2 workers. Max 1 parts (seg0).
8	Planning time: 6.319 ms
9	(slice0) Executor memory: 141K bytes.
10	(slice1) Executor memory: 161K bytes avg x 2 workers, 161K bytes max (seg0).
11	Memory used: 128000kB
12	Optimizer: Pivotal Optimizer (GPORCA)
13	Execution time: 1.593 ms

Postgres Query Optimizer:

📄	EXPLAIN ANALYZE SELECT * FROM companies_partition_dataprekru WHERE dataprekru = '2021-01-15';
⚙️	
Results 1 ×	Output
🔍	EXPLAIN ANALYZE SELECT * FROM companies; Enter a SQL expression to filter results (use Ctrl+Space)
Grid	QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..76.88 rows=21 width=313) (actual time=1.353..1.358 rows=16 loops=1)
2	-> Append (cost=0.00..76.88 rows=11 width=313) (actual time=0.198..0.544 rows=11 loops=1)
3	-> Seq Scan on companies_partition_dataprekru_1_prt_p_default (cost=0.00..19.52 rows=3 width=315) (never executed)
4	Filter: (dataprekru = '2021-01-15'::date)
5	-> Seq Scan on companies_partition_dataprekru_1_prt_p_2001_plus_20 (cost=0.00..57.35 rows=8 width=312) (actual time=0.050..0.393 rows=11 loops=1)
6	Filter: (dataprekru = '2021-01-15'::date)
7	Planning time: 1.560 ms
8	(slice0) Executor memory: 179K bytes.
9	(slice1) Executor memory: 129K bytes avg x 2 workers, 129K bytes max (seg0).
10	Memory used: 128000kB
11	Optimizer: Postgres query optimizer
12	Execution time: 2.460 ms

companies_by_region:

GPORCA:

AI	EXPLAIN ANALYZE SELECT * FROM companies_by_region WHERE dataprekru = '2021-01-15';
📄	
⚙️	
Results 1 ×	Output
🔍	EXPLAIN ANALYZE SELECT * FROM companies; Enter a SQL expression to filter results (use Ctrl+Space)
Grid	QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..441.56 rows=18 width=293) (actual time=6.112..9.012 rows=16 loops=1)
2	-> Seq Scan on companies_by_region (cost=0.00..441.53 rows=9 width=293) (actual time=0.907..5.700 rows=11 loops=1)
3	Filter: (dataprekru = '2021-01-15'::date)
4	Planning time: 5.338 ms
5	(slice0) Executor memory: 69K bytes.
6	(slice1) Executor memory: 65K bytes avg x 2 workers, 65K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Pivotal Optimizer (GPORCA)
9	Execution time: 9.415 ms

Postgres Query Optimizer:

AI	• EXPLAIN ANALYZE SELECT * FROM companies_by_region WHERE dataprekrul = '2021-01-15';
Results 1 x	Output
Grid	EXPLAIN ANALYZE SELECT * FROM companies
Text	Enter a SQL expression to filter results (use Ctrl+Space)
Grid	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=18 width=293) (actual time=11.002..13.292 rows=16 loops=1)
2	-> Seq Scan on companies_by_region (cost=0.00..2262.00 rows=9 width=293) (actual time=1.901..10.459 rows=11 loops=1)
3	Filter: (dataprekrul = '2021-01-15'::date)
4	Planning time: 0.232 ms
5	(slice0) Executor memory: 69K bytes.
6	(slice1) Executor memory: 65K bytes avg x 2 workers, 65K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Postgres query optimizer
9	Execution time: 13.763 ms

Запросы 2:

```
-- 2 запросы
EXPLAIN ANALYZE
SELECT * FROM companies_partition_dataprekrul
WHERE dataprekrul BETWEEN '2018-01-01' AND '2020-12-31';

EXPLAIN ANALYZE
SELECT * FROM companies_by_region
WHERE dataprekrul BETWEEN '2018-01-01' AND '2020-12-31';
```

companies_partition_dataprekrul:

GPORCA:

AI	EXPLAIN ANALYZE SELECT * FROM companies_partition_dataprekrul WHERE dataprekrul BETWEEN '2018-01-01' AND '2020-12-31';
Results 1 x	Output
Grid	EXPLAIN ANALYZE SELECT * FROM companies
Text	Enter a SQL expression to filter results (use Ctrl+Space)
Grid	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..458.70 rows=16869 width=284) (actual time=0.304..8.299 rows=16874 loops=1)
2	-> Sequence (cost=0.00..437.19 rows=8435 width=284) (actual time=0.022..2.121 rows=9424 loops=1)
3	-> Partition Selector for companies_partition_dataprekrul (dynamic scan id: 1) (cost=10.00..100.00 rows=50 width=4) (never executed)
4	Partitions selected: 4 (out of 22)
5	-> Dynamic Seq Scan on companies_partition_dataprekrul (dynamic scan id: 1) (cost=0.00..437.19 rows=8435 width=284) (actual time=0.020..1.733 rows=9424 loops=1)
6	Filter: ((dataprekrul >= '2018-01-01'::date) AND (dataprekrul <= '2020-12-31'::date))
7	Partitions scanned: Avg 4.0 (out of 22) x 2 workers. Max 4 parts (seg0).
8	Planning time: 2.922 ms
9	(slice0) Executor memory: 165K bytes.
10	(slice1) Executor memory: 305K bytes avg x 2 workers, 305K bytes max (seg0).
11	Memory used: 128000kB
12	Optimizer: Pivotal Optimizer (GPORCA)
13	Execution time: 9.262 ms

Postgres Query Optimizer:

<pre>EXPLAIN ANALYZE SELECT * FROM companies_partition_dataprekru1 WHERE dataprekru1 BETWEEN '2018-01-01' AND '2020-12-31';</pre>	
Results 1	Output
EXPLAIN ANALYZE SELECT * FROM companies; Enter a SQL expression to filter results (use Ctrl+Space)	
Grid	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..452.54 rows=16874 width=298) (actual time=0.677..10.371 rows=16874 loops=1)
2	-> Append (cost=0.00..452.54 rows=8437 width=298) (actual time=0.145..3.324 rows=9424 loops=1)
3	-> Seq Scan on companies_partition_dataprekru1_prt_p_default (cost=0.00..21.43 rows=1 width=315) (never executed)
4	Filter: ((dataprekru1 >= '2018-01-01'::date) AND (dataprekru1 <= '2020-12-31'::date))
5	-> Seq Scan on companies_partition_dataprekru1_prt_p_2001_plus_17 (cost=0.00..208.63 rows=4121 width=293) (actual time=0.004..1.354 rows=4745 loops=1)
6	Filter: ((dataprekru1 >= '2018-01-01'::date) AND (dataprekru1 <= '2020-12-31'::date))
7	-> Seq Scan on companies_partition_dataprekru1_prt_p_2001_plus_18 (cost=0.00..132.06 rows=2569 width=301) (actual time=0.006..0.792 rows=2828 loops=1)
8	Filter: ((dataprekru1 >= '2018-01-01'::date) AND (dataprekru1 <= '2020-12-31'::date))
9	-> Seq Scan on companies_partition_dataprekru1_prt_p_2001_plus_19 (cost=0.00..90.42 rows=1748 width=305) (actual time=0.006..0.353 rows=1851 loops=1)
10	Filter: ((dataprekru1 >= '2018-01-01'::date) AND (dataprekru1 <= '2020-12-31'::date))
11	Planning time: 1.218 ms
12	(slice0) Executor memory: 311K bytes.
13	(slice1) Executor memory: 145K bytes avg x 2 workers, 145K bytes max (seg0).
14	Memory used: 128000kB
15	Optimizer: Postgres query optimizer
16	Execution time: 12.063 ms

companies_by_region:

GPORCA:

<pre>EXPLAIN ANALYZE SELECT * FROM companies_by_region WHERE dataprekru1 BETWEEN '2018-01-01' AND '2020-12-31';</pre>	
Results 1	Output
EXPLAIN ANALYZE SELECT * FROM companies; Enter a SQL expression to filter results (use Ctrl+Space)	
Grid	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..469.00 rows=17300 width=293) (actual time=0.425..15.520 rows=16874 loops=1)
2	-> Seq Scan on companies_by_region (cost=0.00..446.24 rows=8650 width=293) (actual time=0.009..9.656 rows=9424 loops=1)
3	Filter: ((dataprekru1 >= '2018-01-01'::date) AND (dataprekru1 <= '2020-12-31'::date))
4	Planning time: 4.427 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Pivotal Optimizer (GPORCA)
9	Execution time: 16.409 ms

Postgres Query Optimizer:

<pre>EXPLAIN ANALYZE SELECT * FROM companies_by_region WHERE dataprekru1 BETWEEN '2018-01-01' AND '2020-12-31';</pre>	
Results 1	Output
EXPLAIN ANALYZE SELECT * FROM companies; Enter a SQL expression to filter results (use Ctrl+Space)	
Grid	AZ QUERY PLAN
1	Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2512.00 rows=17289 width=293) (actual time=0.573..19.371 rows=16874 loops=1)
2	-> Seq Scan on companies_by_region (cost=0.00..2512.00 rows=8645 width=293) (actual time=0.017..11.441 rows=9424 loops=1)
3	Filter: ((dataprekru1 >= '2018-01-01'::date) AND (dataprekru1 <= '2020-12-31'::date))
4	Planning time: 0.285 ms
5	(slice0) Executor memory: 93K bytes.
6	(slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).
7	Memory used: 128000kB
8	Optimizer: Postgres query optimizer
9	Execution time: 20.619 ms

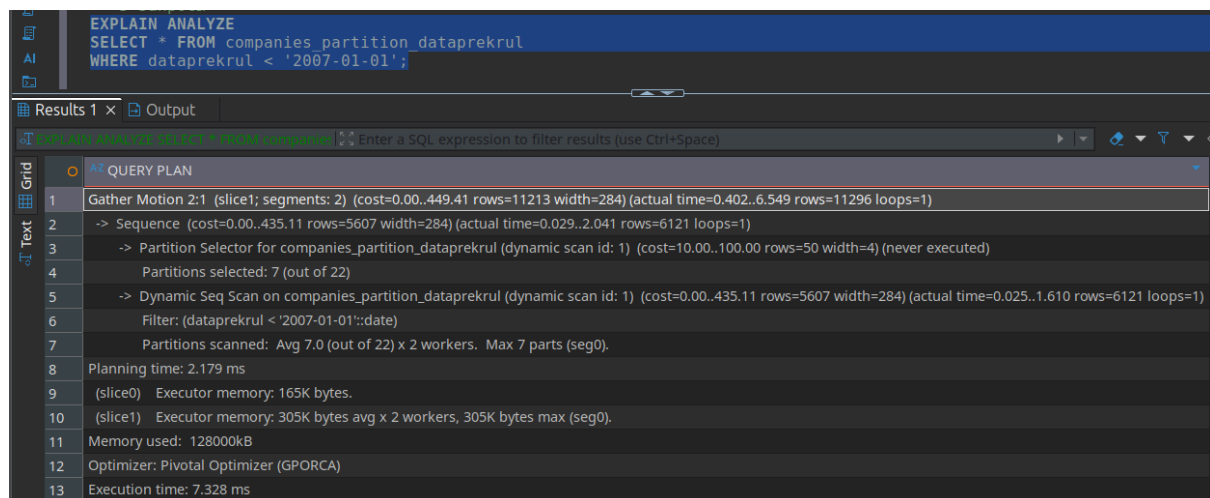
Запросы 3:

```
-- 3 запросы
EXPLAIN ANALYZE
SELECT * FROM companies_partition_dataprekrul
WHERE dataprekrul < '2007-01-01';

EXPLAIN ANALYZE
SELECT * FROM companies_by_region
WHERE dataprekrul < '2007-01-01';
```

companies_partition_dataprekrul:

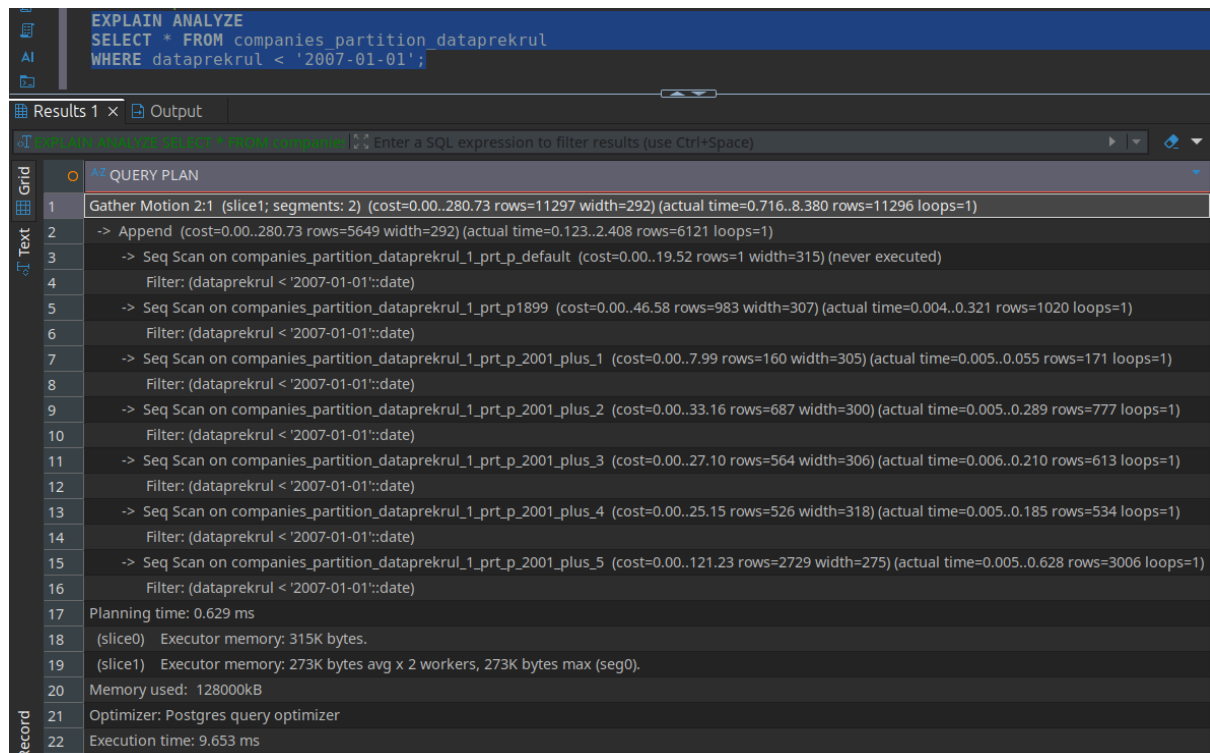
GPORCA:



The screenshot shows the PostgreSQL query planner output for the query `SELECT * FROM companies_partition_dataprekrul WHERE dataprekrul < '2007-01-01';` using the GPORCA optimizer. The query plan is as follows:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Loops
1	Gather Motion 2:1 (slice1; segments: 2)	0.00..449.41	11213	284	0.402..6.549	11296	1
2	-> Sequence	0.00..435.11	5607	284	0.029..2.041	6121	1
3	-> Partition Selector for companies_partition_dataprekrul (dynamic scan id: 1)	10.00..100.00	50	4	(never executed)		
4	Partitions selected: 7 (out of 22)						
5	-> Dynamic Seq Scan on companies_partition_dataprekrul (dynamic scan id: 1)	0.00..435.11	5607	284	0.025..1.610	6121	1
6	Filter: (dataprekrul < '2007-01-01'::date)						
7	Partitions scanned: Avg 7.0 (out of 22) x 2 workers. Max 7 parts (seg0).						
8	Planning time: 2.179 ms						
9	(slice0) Executor memory: 165K bytes.						
10	(slice1) Executor memory: 305K bytes avg x 2 workers, 305K bytes max (seg0).						
11	Memory used: 128000kB						
12	Optimizer: Pivotal Optimizer (GPORCA)						
13	Execution time: 7.328 ms						

Postgres Query Optimizer:



The screenshot shows the PostgreSQL query planner output for the same query using the Postgres query optimizer. The query plan is as follows:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Loops
1	Gather Motion 2:1 (slice1; segments: 2)	0.00..280.73	11297	292	0.716..8.380	11296	1
2	-> Append	0.00..280.73	5649	292	0.123..2.408	6121	1
3	-> Seq Scan on companies_partition_dataprekrul_1_prt_p_default	0.00..19.52	1	315	(never executed)		
4	Filter: (dataprekrul < '2007-01-01'::date)						
5	-> Seq Scan on companies_partition_dataprekrul_1_prt_p1899	0.00..46.58	983	307	0.004..0.321	1020	1
6	Filter: (dataprekrul < '2007-01-01'::date)						
7	-> Seq Scan on companies_partition_dataprekrul_1_prt_p_2001_plus_1	0.00..7.99	160	305	0.005..0.055	171	1
8	Filter: (dataprekrul < '2007-01-01'::date)						
9	-> Seq Scan on companies_partition_dataprekrul_1_prt_p_2001_plus_2	0.00..33.16	687	300	0.005..0.289	777	1
10	Filter: (dataprekrul < '2007-01-01'::date)						
11	-> Seq Scan on companies_partition_dataprekrul_1_prt_p_2001_plus_3	0.00..27.10	564	306	0.006..0.210	613	1
12	Filter: (dataprekrul < '2007-01-01'::date)						
13	-> Seq Scan on companies_partition_dataprekrul_1_prt_p_2001_plus_4	0.00..25.15	526	318	0.005..0.185	534	1
14	Filter: (dataprekrul < '2007-01-01'::date)						
15	-> Seq Scan on companies_partition_dataprekrul_1_prt_p_2001_plus_5	0.00..121.23	2729	275	0.005..0.628	3006	1
16	Filter: (dataprekrul < '2007-01-01'::date)						
17	Planning time: 0.629 ms						
18	(slice0) Executor memory: 315K bytes.						
19	(slice1) Executor memory: 273K bytes avg x 2 workers, 273K bytes max (seg0).						
20	Memory used: 128000kB						
21	Optimizer: Postgres query optimizer						
22	Execution time: 9.653 ms						

companies_by_region:
GPORCA:

EXPLAIN ANALYZE
SELECT * FROM companies_by_region
WHERE dataprekru1 < '2007-01-01';

Results 1 × Output

EXPLAIN ANALYZE SELECT * FROM companies

Grid

Text

AZ QUERY PLAN

1 Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..459.68 rows=11434 width=293) (actual time=0.292..10.624 rows=11296 loops=1)

2 -> Seq Scan on companies_by_region (cost=0.00..444.64 rows=5717 width=293) (actual time=0.010..7.659 rows=6121 loops=1)

3 Filter: (dataprekru1 < '2007-01-01'::date)

4 Planning time: 19.533 ms

5 (slice0) Executor memory: 93K bytes.

6 (slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).

7 Memory used: 128000kB

8 Optimizer: Pivotal Optimizer (GPORCA)

9 Execution time: 11.216 ms

Postgres Query Optimizer:

EXPLAIN ANALYZE
SELECT * FROM companies_by_region
WHERE dataprekru1 < '2007-01-01';

Results 1 × Output

EXPLAIN ANALYZE SELECT * FROM companies

Grid

Text

AZ QUERY PLAN

1 Gather Motion 2:1 (slice1; segments: 2) (cost=0.00..2262.00 rows=11433 width=293) (actual time=0.520..17.959 rows=11296 loops=1)

2 -> Seq Scan on companies_by_region (cost=0.00..2262.00 rows=5717 width=293) (actual time=0.015..12.331 rows=6121 loops=1)

3 Filter: (dataprekru1 < '2007-01-01'::date)

4 Planning time: 0.216 ms

5 (slice0) Executor memory: 93K bytes.

6 (slice1) Executor memory: 81K bytes avg x 2 workers, 81K bytes max (seg0).

7 Memory used: 128000kB

8 Optimizer: Postgres query optimizer

9 Execution time: 19.178 ms

Результат в виде таблицы:

Номер запроса	Таблица	Оптимизатор	Время планирования	Время исполнения
1 = '2021-01-15'	companies_partition_dataprekru1	GPORCA	6.319 ms	1.593 ms
1 = '2021-01-15'	companies_partition_dataprekru1	Postgres Query Optimizer	1.560 ms	2.460 ms
1 = '2021-01-15'	companies_by_region	GPORCA	5.338 ms	9.415 ms
1	companies_by_r	Postgres Query	0.232 ms	13.763 ms

= '2021-01-15'	egion	Optimizer		
2 BETWEEN '2018-01-01' AND '2020-12-31'	companies_parti tion_dataprekrul	GPORCA	2.922 ms	9.262 ms
2 BETWEEN '2018-01-01' AND '2020-12-31'	companies_parti tion_dataprekrul	Postgres Query Optimizer	1.218 ms	12.063 ms
2 BETWEEN '2018-01-01' AND '2020-12-31'	companies_by_r egion	GPORCA	4.427 ms	16.409 ms
2 BETWEEN '2018-01-01' AND '2020-12-31'	companies_by_r egion	Postgres Query Optimizer	0.285 ms	20.619 ms
3 < '2007-01-01'	companies_parti tion_dataprekrul	GPORCA	2.179 ms	7.328 ms
3 < '2007-01-01'	companies_parti tion_dataprekrul	Postgres Query Optimizer	0.629 ms	9.653 ms
3 < '2007-01-01'	companies_by_r egion	GPORCA	19.533 ms	11.216 ms
3 < '2007-01-01'	companies_by_r egion	Postgres Query Optimizer	0.216 ms	19.178 ms

Вывод:

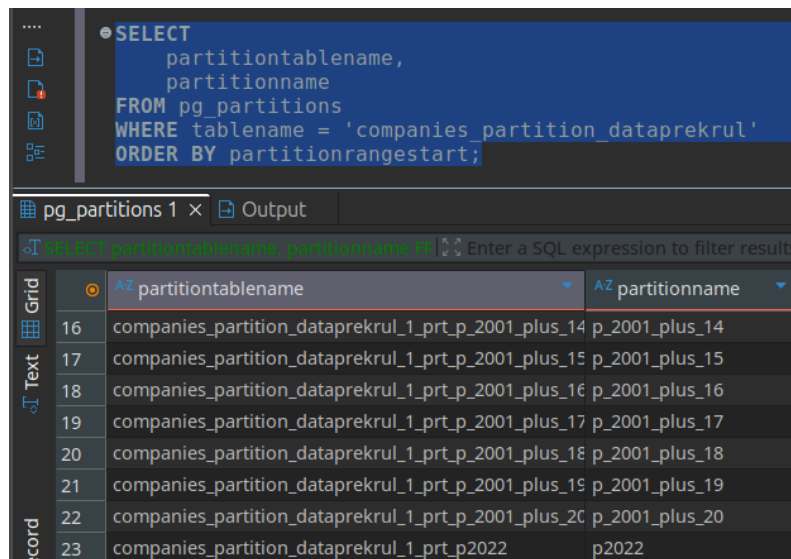
Здесь также можно увидеть, что партицирование дает небольшое ускорение, но так как данных мало, то оно небольшое. Если бы данных было много, то эффект был бы намного нагляднее. Но даже так все равно можно увидеть разницу и понять эффективность партицирования.

9. Выполнить расщепление партиции по умолчанию для таблицы из предыдущего пункта для выделения в отдельную партицию записей за 2022 год.

Выполним расщепление используя следующий скрипт:

```
ALTER TABLE companies_partition_dataprekru1
SPLIT DEFAULT PARTITION
START ('2022-01-01') INCLUSIVE
END ('2023-01-01') EXCLUSIVE
INTO (PARTITION p2022, DEFAULT PARTITION);
```

Проверим, что партиция появилась:



```
SELECT
    partitiontablename,
    partitionname
FROM pg_partitions
WHERE tablename = 'companies_partition_dataprekru1'
ORDER BY partitionrangestart;
```

	AZ partitiontablename	AZ partitionname
16	companies_partition_dataprekru1_prt_p_2001_plus_14	p_2001_plus_14
17	companies_partition_dataprekru1_prt_p_2001_plus_15	p_2001_plus_15
18	companies_partition_dataprekru1_prt_p_2001_plus_16	p_2001_plus_16
19	companies_partition_dataprekru1_prt_p_2001_plus_17	p_2001_plus_17
20	companies_partition_dataprekru1_prt_p_2001_plus_18	p_2001_plus_18
21	companies_partition_dataprekru1_prt_p_2001_plus_19	p_2001_plus_19
22	companies_partition_dataprekru1_prt_p_2001_plus_20	p_2001_plus_20
23	companies_partition_dataprekru1_prt_p2022	p2022

Все получилось!

Дополнительные задания

10. Оцените равномерность распределение записей по партициям (для обеих таблиц).

Оценка равномерности распределения записей по партициям таблицы companies_partition_list:

```
WITH stats AS (
    SELECT 'p77' as partition_name, COUNT(*) as row_count
    FROM companies_partition_list_1_prt_r77
    UNION ALL
    SELECT 'p78', COUNT(*) FROM companies_partition_list_1_prt_r78
    UNION ALL
    SELECT 'p66', COUNT(*) FROM companies_partition_list_1_prt_r66
    UNION ALL
    SELECT 'p50', COUNT(*) FROM companies_partition_list_1_prt_r50
    UNION ALL
    SELECT 'r_other', COUNT(*) FROM companies_partition_list_1_prt_r_other
)
SELECT
    partition_name,
    row_count,
```

```
ROUND(100.0 * row_count / SUM(row_count) OVER (), 2) as percentage
FROM stats;
```

Результат:

	partition_name	row_count	percentage
1	p77	25,876	25.88
2	p78	9,033	9.03
3	p66	3,891	3.89
4	p50	3,668	3.67
5	r_other	57,532	57.53

Очевидно, что партии распределены не равномерно, есть перекоc - в p77 26% всех данных, а в r_other 58%

Оценка равномерности распределения записей по партициям таблицы companies_partition_dataprekrul:

```
WITH stats AS (
    SELECT 'p_default' as partition_name, COUNT(*) as row_count FROM
companies_partition_dataprekrul_1 prt p_default
    UNION ALL SELECT 'p1899', COUNT(*) FROM
companies_partition_dataprekrul_1 prt p1899
    UNION ALL SELECT 'p_2001_plus_1', COUNT(*) FROM
companies_partition_dataprekrul_1 prt p_2001_plus_1
    UNION ALL SELECT 'p_2001_plus_2', COUNT(*) FROM
companies_partition_dataprekrul_1 prt p_2001_plus_2
    UNION ALL SELECT 'p_2001_plus_3', COUNT(*) FROM
companies_partition_dataprekrul_1 prt p_2001_plus_3
    UNION ALL SELECT 'p_2001_plus_4', COUNT(*) FROM
companies_partition_dataprekrul_1 prt p_2001_plus_4
    UNION ALL SELECT 'p_2001_plus_5', COUNT(*) FROM
companies_partition_dataprekrul_1 prt p_2001_plus_5
    UNION ALL SELECT 'p_2001_plus_6', COUNT(*) FROM
companies_partition_dataprekrul_1 prt p_2001_plus_6
    UNION ALL SELECT 'p_2001_plus_7', COUNT(*) FROM
companies_partition_dataprekrul_1 prt p_2001_plus_7
    UNION ALL SELECT 'p_2001_plus_8', COUNT(*) FROM
companies_partition_dataprekrul_1 prt p_2001_plus_8
    UNION ALL SELECT 'p_2001_plus_9', COUNT(*) FROM
companies_partition_dataprekrul_1 prt p_2001_plus_9
    UNION ALL SELECT 'p_2001_plus_10', COUNT(*) FROM
companies_partition_dataprekrul_1 prt p_2001_plus_10
    UNION ALL SELECT 'p_2001_plus_11', COUNT(*) FROM
companies_partition_dataprekrul_1 prt p_2001_plus_11
    UNION ALL SELECT 'p_2001_plus_12', COUNT(*) FROM
companies_partition_dataprekrul_1 prt p_2001_plus_12
    UNION ALL SELECT 'p_2001_plus_13', COUNT(*) FROM
companies_partition_dataprekrul_1 prt p_2001_plus_13
```

```

        UNION ALL SELECT 'p_2001_plus_14', COUNT(*) FROM
companies_partition_dataprekrul_1_prt_p_2001_plus_14
        UNION ALL SELECT 'p_2001_plus_15', COUNT(*) FROM
companies_partition_dataprekrul_1_prt_p_2001_plus_15
        UNION ALL SELECT 'p_2001_plus_16', COUNT(*) FROM
companies_partition_dataprekrul_1_prt_p_2001_plus_16
        UNION ALL SELECT 'p_2001_plus_17', COUNT(*) FROM
companies_partition_dataprekrul_1_prt_p_2001_plus_17
        UNION ALL SELECT 'p_2001_plus_18', COUNT(*) FROM
companies_partition_dataprekrul_1_prt_p_2001_plus_18
        UNION ALL SELECT 'p_2001_plus_19', COUNT(*) FROM
companies_partition_dataprekrul_1_prt_p_2001_plus_19
        UNION ALL SELECT 'p_2001_plus_20', COUNT(*) FROM
companies_partition_dataprekrul_1_prt_p_2001_plus_20
        UNION ALL SELECT 'p2022', COUNT(*) FROM
companies_partition_dataprekrul_1_prt_p2022
    )
SELECT
    partition_name,
    row_count,
    ROUND(100.0 * row_count / SUM(row_count) OVER (), 2) as percentage
FROM stats
ORDER BY partition_name;

```

Результат:

	AZ partition_name	row_count	percentage
1	p1899	1,966	1.97
2	p_2001_plus_1	319	0.32
3	p_2001_plus_10	7,754	7.75
4	p_2001_plus_11	7,762	7.76
5	p_2001_plus_12	6,256	6.26
6	p_2001_plus_13	6,738	6.74
7	p_2001_plus_14	4,450	4.45
8	p_2001_plus_15	10,031	10.03
9	p_2001_plus_16	8,390	8.39
10	p_2001_plus_17	8,242	8.24
11	p_2001_plus_18	5,137	5.14
12	p_2001_plus_19	3,495	3.5
13	p_2001_plus_2	1,373	1.37
14	p_2001_plus_20	2,428	2.43
15	p_2001_plus_3	1,128	1.13
16	p_2001_plus_4	1,052	1.05
17	p_2001_plus_5	5,458	5.46
18	p_2001_plus_6	6,674	6.67
19	p_2001_plus_7	3,812	3.81
20	p_2001_plus_8	3,382	3.38
21	p_2001_plus_9	3,391	3.39
22	p2022	762	0.76
23	p_default	0	0

Здесь уже можно сказать, что партии распределены очень равномерно. Нет явного перекоса, что очень хорошо и эффективно при отсечении партий, ведь получится отсекать большое количество данных.

11. Разработать «неправильную» схему секционирования, при которой при добавлении записи выдавалась бы ошибка, обусловленная неправильным секционированием. Исправьте эту ошибку.

Создадим такую схему:

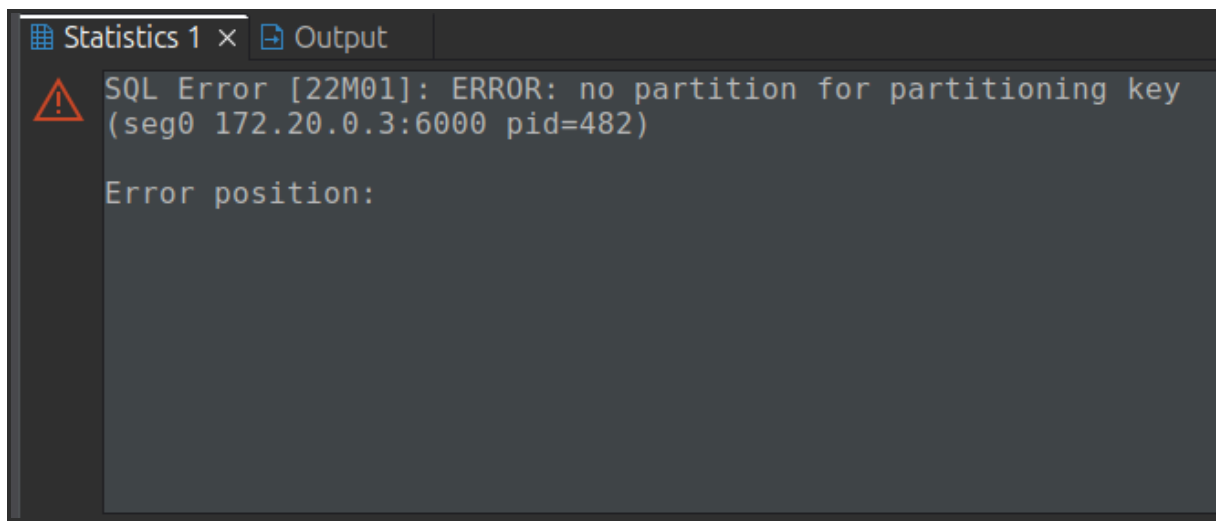
```
CREATE TABLE companies_partition_bad (
  LIKE companies
)
DISTRIBUTED BY (region_id)
PARTITION BY RANGE (dataprekrul)
(
  PARTITION p2015 START ('2015-01-01') END ('2016-01-01'),
```

```
PARTITION p2016 START ('2016-01-01') END ('2017-01-01'),  
PARTITION p2017 START ('2017-01-01') END ('2018-01-01')  
);
```

Проблема в том, что партиции не покрывают все года. А дефолтной партиции нет.

Так что при такой вставке будет ошибка:

```
INSERT INTO companies_partition_bad SELECT * FROM companies;
```



Можно исправить следующим образом:

```
ALTER TABLE companies_partition_bad  
ADD DEFAULT PARTITION p_default;
```

То есть просто добавили дефолтную партицию куда будут попадать все остальные записи для которых нет нужных годов:

```
INSERT INTO companies_partition_bad SELECT * FROM companies;
```

